

A Branch-and-Cut Algorithm for the Resolution of Large-Scale Symmetric Traveling Salesman Problems

Manfred Padberg; Giovanni Rinaldi

SIAM Review, Vol. 33, No. 1 (Mar., 1991), 60-100.

Stable URL:
http://links.jstor.org/sici?sici=0036-1445%28199103%2933%3A1%3C60%3AABAFTR%3E2.0.CO%3B2-T

SIAM Review is currently published by Society for Industrial and Applied Mathematics.

Your use of the JSTOR archive indicates your acceptance of JSTOR's Terms and Conditions of Use, available at http://www.jstor.org/about/terms.html. JSTOR's Terms and Conditions of Use provides, in part, that unless you have obtained prior permission, you may not download an entire issue of a journal or multiple copies of articles, and you may use content in the JSTOR archive only for your personal, non-commercial use.

Please contact the publisher regarding any further use of this work. Publisher contact information may be obtained at http://www.jstor.org/journals/siam.html.

Each copy of any part of a JSTOR transmission must contain the same copyright notice that appears on the screen or printed page of such transmission.

JSTOR is an independent not-for-profit organization dedicated to creating and preserving a digital archive of scholarly journals. For more information regarding JSTOR, please contact support@jstor.org.

http://www.jstor.org/
Thu Mar 17 15:45:10 2005

SIAM REVIEW
Vol. 33, No. 1, pp. 60–100, March 1991
©1991 Society for Industrial and Applied Mathematics
004

A BRANCH-AND-CUT ALGORITHM FOR THE RESOLUTION OF LARGE-SCALE SYMMETRIC TRAVELING SALESMAN PROBLEMS *

MANFRED PADBERG† AND GIOVANNI RINALDI‡

Abstract. An algorithm is described for solving large-scale instances of the Symmetric Traveling Salesman Problem (STSP) to optimality. The core of the algorithm is a "polyhedral" cutting-plane procedure that exploits a subset of the system of linear inequalities defining the convex hull of the incidence vectors of the hamiltonian cycles of a complete graph. The cuts are generated by several identification procedures that have been described in a companion paper. Whenever the cutting-plane procedure does not terminate with an optimal solution the algorithm uses a tree-search strategy that, as opposed to branch-and-bound, keeps on producing cuts after branching. The algorithm has been implemented in FORTRAN. Two different linear programming (LP) packages have been used as the LP solver. The implementation of the algorithm and the interface with one of the LP solvers is described in sufficient detail to permit the replication of our experiments. Computational results are reported with up to 42 STSPs with sizes ranging from 48 to 2,392 nodes. Most of the medium-sized test problems are taken from the literature; all others are large-scale real-world problems. All of the instances considered in this study were solved to optimality by the algorithm in "reasonable" computation times.

Key words. symmetric traveling salesman problem, branch-and-cut, scientific computation, polyhedral theory, facets, cutting planes, heuristics, software development

AMS(MOS) subject classifications. 90C10, 05C35, 68E10, 52A40

1. Introduction.

Let $K_n = (V, E)$ be the complete undirected graph with $n = |V|$ nodes and $m = |E| = \binom{n}{2}$ edges. A *tour* is a subset of E given by a *Hamiltonian cycle* of K_n. An edge e with endpoints u and v is denoted by $[u, v]$. Given a set $W \subseteq V$ we denote by $E(W)$ the subset of E defined as $\{[u, v] \in E \mid u, v \in W\}$. We denote by $\mathbb{R}^E$ the space of real vectors whose components are indexed by the elements of E. The component of a vector $x \in \mathbb{R}^E$ indexed by the edge $e = [u, v]$ is denoted by x_e or $x(u, v)$, and for any edge set $F \subseteq E$ we denote by $x(F)$ the sum $\sum_{e \in F} x_e$.

With every edge $e \in E$ we associate a rational number c_e, which is interpreted as the "length" of the edge e. Denoting $c \in \mathbb{R}^E$ the resulting vector we have for every tour τ in K_n an associated "tour length" $c(\tau)$. The *Symmetric Traveling Salesman Problem* (STSP) consists of finding a tour in K_n of minimum length and is one of the best known and most studied combinatorial optimization problems. Most of the exact or heuristic techniques for the solution of "hard" combinatorial problems have been thought of, developed for, and tried out on the STSP, which—as a result—has become a kind of *prototype* problem in the combinatorial optimization literature (see Hoffman and Wolfe (1985)). This fact is appropriately reflected by the title *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization,* of the book edited

*Received by the editors April 24, 1989; accepted for publication (in revised form) June 7, 1990.

†New York University, Washington Square, New York, New York 10003. Partial financial support by National Science Foundation grant DMS8508955 and Office of Naval Research grant R&T4116663.

‡Istituto di Analisi dei Sistemi ed Informatica del Consiglio Nazionale delle Ricerche, Viale Manzoni 30, 00185 Roma, Italy. Partial financial support under a New York University Research Challenge Grant (1986), Office of Naval Research grant R&T4116663 and IBM I-U Agreement No. 10840056.

by Lawler et al. (1985) which contains a wide selection of surveys on complexity issues, problem formulations, and the techniques developed to provide "good" solutions to the problem or to solve it to optimality. We refer the reader to this book for the basic background on the problem and the techniques for its solution.

As far as the solution to optimality is concerned, the most successful technique is the one based on the linear description of the STSP polytope. This technique has its roots in the seminal papers by Dantzig, Fulkerson, and Johnson (1954) and (1959), where the solution of a 49-city problem and of a 10-city problem, respectively, are reported. After an in-depth study of the facial structure of the polytope associated with the STSP by Grötschel and Padberg (1979a), (1979b), the technique was enhanced substantially and shown to allow the solution of much larger problems: In Grötschel (1977), (1980) a 120-city problem is solved with an interactive procedure; in Padberg and Hong (1980) an automatic procedure is described that solved problems with up to 120 cities; see Grötschel and Padberg (1978) for a synopsis of these results. Finally, in Crowder and Padberg (1980) this technique is used to provide a commercial branch-and-bound algorithm with a strong lower bound. In this way 10 large-scale problems with sizes ranging from 48 to 318 cities were solved to optimality. For a complete survey on the linear programming approach to the solution of STSPs and related computational studies see Grötschel and Padberg (1985) and Padberg and Grötschel (1985).

Until the publication of Padberg and Rinaldi (1987) where some of the results of the computational study described here are reported in preliminary form, the 318-city problem was, to the best of our knowledge of the literature, the largest instance of an STSP solved to optimality. We began the work described here in 1985 in the hope of being able to demonstrate that the approach based on the polyhedral description of combinatorial optimization problems can be successfully exploited to solve problem instances even one order of magnitude bigger than those solved previously to optimality. And this is indeed the case, even though the polyhedral description of the problem is far from being complete. The largest instance that we have solved has 2,392 nodes and the corresponding integer linear programming formulation has 2,859,636 zero-one variables and, a priori, an equally huge amount of constraints. Evidently, to solve problems of this size in execution times and with memory allocations that are reasonable for today's computers, much has to be done on the implementation side and everything that can be of any help is borrowed from other techniques used in combinatorial optimization. There are four key elements that characterize the algorithm described here: A heuristic procedure to find a good upper bound on the optimal tour length, a set of procedures for the identification of violated inequalities of the partial description of the STSP polytope, a carefully designed interface with the LP solver, and a tree-search approach that combines branching with cutting-plane techniques. Recently, Grötschel and Holland (1988) completed a numerical study of the STSP in a similar vein and reported the optimal solution of problem instances for the STSP with up to 1,000 nodes. In Fig. 1 and Table 1 we summarize the history of exact computation of STSPs to the extent that it is known to us.

In § 2 we describe the whole algorithm and the branch-and-cut approach. In § 3 we briefly outline the heuristic we use to find a good integer feasible solution of the problem and to provide the algorithm with a good upper bound and a good starting set of "active" variables. In § 4 we describe the constraint generation procedure. In § 5 we survey many implementational details of the interface with the LP solver. In § 6 we describe the implementation of branch-and-cut as a tree-search procedure and finally in § 7 we report our computational experience with the algorithm.

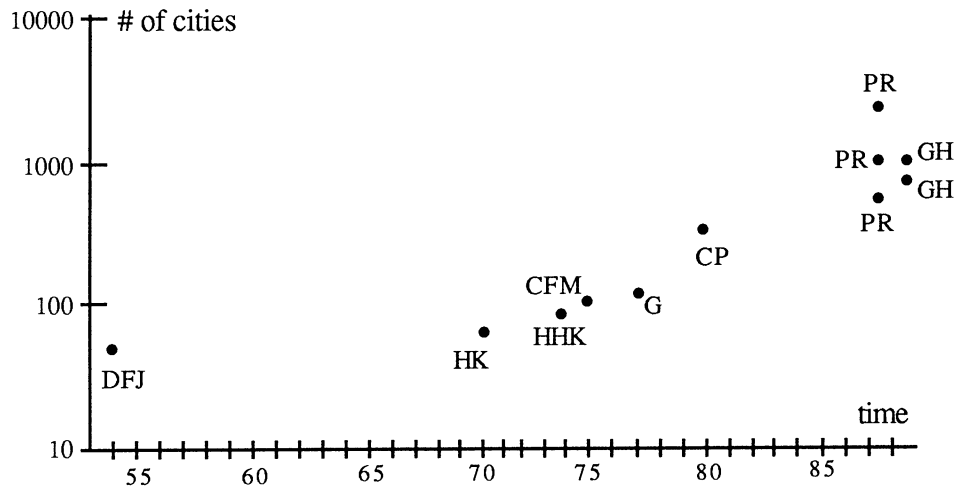


FIG. 1. *History of exact optimization of* STSPs.

TABLE 1

Year		Reference	# of cities	# of 0–1 var.
1954	DFJ:	Dantzig, Fulkerson and Johnson	49	1,076
1970	HK:	Held and Karp	60	1,770
1974	HHK:	Helbig Hansen and Krarup	80	3,160
1975	CFM:	Camerini, Fratta and Maffioli	100	4,950
1977	G:	Grötschel	120	7,140
1980	CP:	Crowder and Padberg	318	50,403
1987	PR:	Padberg and Rinaldi	532	141,246
1987	PR:	Padberg and Rinaldi*	1,002	501,501
1987	PR:	Padberg and Rinaldi*	2,392	2,859,636
1988	GH:	Grötschel and Holland	666	221,445
1988	GH:	Grötschel and Holland	1,000	499,500

* Note added in proof.

Throughout the paper we assume that the reader is familiar with linear programming, especially the primal and dual simplex method, as well as the fundamentals of graph theory.

2. The branch-and-cut algorithm. With every tour τ of K_n, we associate an *incidence vector* $x^\tau \in R^E$ with components

$$x_e^\tau = \begin{cases} 1 & \text{if } e \in \tau, \\ 0 & \text{if } e \notin \tau. \end{cases}$$

The STSP polytope Q^n (or Q^V) is the convex hull of the incidence vectors of all tours of K_n:

$$Q^n = \operatorname{conv}\left\{x^\tau \in \mathbb{R}^E \mid \tau \text{ is a tour of } K_n\right\}.$$

Since every node is met by exactly two edges of a tour, Q^n is contained in the polytope

$$Q_A^n = \left\{x \in \mathbb{R}^E \mid Ax = \mathbf{2}, \mathbf{0} \le x \le \mathbf{1}\right\},$$

where A is the node-edge incidence matrix of K_n, $\mathbf{0}$, $\mathbf{1}$, and $\mathbf{2}$ are vectors having all components equal to 0, 1 and 2, respectively. The equations $Ax = \mathbf{2}$ are called the *degree equations*. By a theorem of Weyl (1935), Q^n can be described by a system of linear inequalities, i.e.,

$$Q^n = \left\{ x \in Q_A^n \mid lx \leq l_0 \quad \text{for all } (l, l_0) \in \mathcal{L}_n \right\},$$

where $\mathcal{L}_n$ is a finite family of linear inequalities. Moreover, $\mathcal{L}_n$ can be chosen such that each inequality of $\mathcal{L}_n$ induces a different facet of Q^n. For the definition of a *facet* and for all basic concepts of polyhedral theory we refer the reader to Grötschel and Padberg (1985).

2.1. Polyhedral cutting-plane algorithms. The optimal solution of STSP is given by a minimum length tour and can be found by solving the linear program

$$\begin{aligned} \min \quad & cx \\ \text{s.t.} \quad & Ax = \mathbf{2}, \\ & lx \leq l_0 \quad \text{for all } (l, l_0) \in \mathcal{L}_n, \\ & \mathbf{0} \leq x \leq \mathbf{1}. \end{aligned} \tag{2.1}$$

Only a proper subfamily $\overline{\mathcal{L}}$ of the family $\mathcal{L}_n$ of facet-inducing inequalities is known for Q^n at present. Therefore we solve the following relaxed problem:

$$\begin{aligned} \min \quad & cx \\ \text{s.t.} \quad & Ax = \mathbf{2}, \\ & lx \leq l_0 \quad \text{for all } (l, l_0) \in \overline{\mathcal{L}}, \\ & \mathbf{0} \leq x \leq \mathbf{1}. \end{aligned} \tag{2.2}$$

Its solution x^* is either the incidence vector of a tour or it violates some unknown inequality contained in $\mathcal{L}_n - \overline{\mathcal{L}}$. In the first case, we have solved (2.1). In the second case, we have a lower bound on the optimal value of (2.1) that can be used in a tree-search algorithm to find an optimal solution.

The subfamily $\overline{\mathcal{L}}$ contains a superexponential number of inequalities and hence it is impossible to solve (2.2) by giving an *explicit* list of all of them. Yet (2.2) can be solved by the following general *polyhedral* cutting-plane procedure, which uses facet-defining inequalities in lieu of the traditional textbook cutting planes (see Padberg and Rao (1982)):

PROCEDURE 2.1. *Input:* n, c, a family of "known" inequalities $\mathcal{L}'$.

Step 1. Set $\mathcal{L} = \emptyset$.

Step 2. Solve the problem (2.2) and let $\overline{x}$ be its solution.

Step 3. Find one or more inequalities in $\mathcal{L}'$ that are violated by $\overline{x}$.

Step 4. If none is found, stop. Otherwise add the violated inequalities to $\mathcal{L}$ and go to Step 2.

Procedure 2.1 stops after a finite number of steps, because $\mathcal{L}'$ is finite. The *core* of the procedure is Step 3, which is called the *constraint identification problem* (or *separation problem*) and which is formally stated as follows.

PROBLEM 2.2. Given a point $x \in \mathbb{R}^E$ and a family $\mathcal{L}'$ of inequalities in $\mathbb{R}^E$, *identify* one or more inequalities in $\mathcal{L}'$ that is violated by x or prove that no such inequality exists.

Given a family of inequalities $\mathcal{L}'$, we call a procedure that *solves* Problem 2.2 *exact*, and we call a procedure that sometimes identifies violated inequalities, but that does not guarantee the solution of Problem 2.2, *heuristic*.

The faster, the more efficient, and the more "prolific" the identification procedure used in Step 3 of Procedure 2.1, the faster is the resolution of (2.2) or any other combinatorial optimization problem with a linear objective function. This statement is substantiated by a result of Padberg and Rao (1981) and Grötschel, Lovász, and Schrijver (1981), which states that (2.2) can be solved in polynomial time *if and only if* Problem 2.2 can be solved in polynomial time. The first class of integer programming problems for which the polyhedral cutting-plane Procedure 2.1 was shown to work *always* is the class of b-matching problems; see Padberg and Rao (1982).

2.2. The classical approach. Many families of inequalities that induce facets of Q^n are known to date; see Grötschel and Padberg (1985) and the more recent results by Boyd and Cunningham (1988) and Naddef and Rinaldi (1988a–d). In Padberg and Rinaldi (1990a) we consider a family $\mathcal{L}'$ that contains the *subtour elimination*, the 2-*matching*, the *comb*, and the *clique-tree* inequalities. For these inequalities we propose several exact and heuristic identification procedures which are designed to handle graphs with a large number of nodes. Since some of these procedures are not exact, Procedure 2.1 terminates either because the current solution $\overline{x}$ satisfies all inequalities in $\mathcal{L}'$ or because the identification procedure is not able to identify an inequality of $\mathcal{L}'$ that is violated by $\overline{x}$.

If Procedure 2.1 terminates with a vector $\overline{x}$ that is not the incidence vector of a tour, then it is necessary to resort to some sort of enumeration to solve the problem to optimality. The by now "classical" approach used first in Crowder and Padberg (1980) and surveyed in Hoffman and Padberg (1985/6) goes as follows: The linear program (2.1) is equivalent to the *integer* linear program

$$
\begin{aligned}
\min \quad & cx \\
\text{s.t.} \quad & Ax = \mathbf{2}, \\
& x(E(W)) \le |W| - 1 \quad \text{for all } \emptyset \subset W \subset V, \\
& \mathbf{0} \le x \le \mathbf{1}, \\
& x \text{ integer},
\end{aligned}
\tag{2.3}
$$

where, besides the degree equations and the upper bounds, only the subtour elimination inequalities are used. To solve the STSP (2.1) the linear program *amended* by the inequalities in the set $\mathcal{L}$ of Procedure 2.1 is taken as the linear programming relaxation of the problem (2.3), which is subsequently subjected to a commercial branch-and-bound algorithm such as IBM's MPSX-MIP/370. Thus the solution x^* found by branch-and-bound is integer and satisfies all inequalities of $\mathcal{L}$. If x^* is the incidence vector of a tour it is also the optimal solution to (2.3). Otherwise it is the incidence vector of a set of subtours. In this case one or more subtour elimination inequalities are added which are violated by x^* to $\mathcal{L}$ and the branch-and-bound algorithm is restarted from scratch with the augmented linear programming relaxation. Eventually the solution x^* is the incidence vector of a tour and the process terminates. The Grötschel and Holland (1988) study follows this methodology, and for more details on this procedure we refer the reader to the book by Nemhauser and Wolsey (1988), where this algorithm is called the *Fractional Cutting-Plane Algorithm* (FCPA).

2.3. The branch-and-cut approach. The advantage of the classical approach to combinatorial optimization problems, for which only a partial description of the

associated polyhedron is known, is the fact that any commercial branch-and-bound code can be used in the enumeration phase. On the other hand, there are at least two drawbacks to this approach, which are due to the inflexibility of the existing branch-and-bound methodology, and which stimulated us to pursue an alternative approach: (a) Since the cutting-plane phase and the enumeration phase are completely separated in a branch-and-bound algorithm, new information about the known partial linear description of Q^n cannot be exploited at all during the enumeration phase. (b) If the branch-and-bound algorithm terminates with a subtour solution, the entire enumeration process has to be repeated from scratch, which is wasteful.

To overcome these two drawbacks we use a different approach that consists of integrating the cutting plane with the enumeration phase. In Padberg and Rinaldi (1987) we have termed this approach *branch-and-cut* and it goes as follows: Given a set $\mathcal{L} \subset \mathcal{L}_n$ of inequalities that are facet-inducing for Q^n and two disjoint edge sets F_0, $F_1 \subset E$, we denote by $P(\mathcal{L}, F_0, F_1)$ the linear program

$$
\begin{aligned}
P(\mathcal{L}, F_0, F_1): \quad \min \quad & cx \\
\text{s.t.} \quad & Ax = \mathbf{2}, \\
& lx \le l_0 \quad \text{for all } (l, l_0) \in \mathcal{L}, \\
& x_e = 0 \quad \text{for all } e \in F_0, \\
& x_e = 1 \quad \text{for all } e \in F_1, \\
& \mathbf{0} \le x \le \mathbf{1}.
\end{aligned} \tag{2.4}
$$

By $\mathcal{S}$ we denote a family of ordered pairs of disjoint edge sets, $\langle F_0, F_1 \rangle$ denotes an ordered pair in $\mathcal{S}$, and x^* is the incidence vector of some tour of K_n.

ALGORITHM 2.3 (BRANCH-AND-CUT). *Input: n, c, x^*.*

Step 0. Set $\mathcal{S} = \{\langle \emptyset, \emptyset \rangle\}$, $\mathcal{L} = \emptyset$.

Step 1. If $\mathcal{S} = \emptyset$, then stop. Otherwise pick an ordered pair $\langle F_0, F_1 \rangle$ from $\mathcal{S}$ and replace $\mathcal{S}$ by $\mathcal{S} - \langle F_0, F_1 \rangle$.

Step 2. Solve the linear program $P(\mathcal{L}, F_0, F_1)$. If the program is inconsistent, go to Step 1. Otherwise let $\overline{x}$ be its optimal solution.

Step 3. If $c\overline{x} \ge cx^*$, go to Step 1.

Step 4. Find one or more inequalities of $\mathcal{L}_n$ that are violated by $\overline{x}$.

Step 5. If none is found, go to Step 6. Otherwise add the violated inequalities to $\mathcal{L}$ and go to Step 2.

Step 6. If $\overline{x}$ is integer, then replace x^* by $\overline{x}$ and go to Step 1.

Step 7. Pick an edge $e \in E$ such that $0 < \overline{x}_e < 1$. Replace $\mathcal{S}$ by $\mathcal{S} + \langle F_0 + \{e\}, F_1 \rangle + \langle F_0, F_1 + \{e\} \rangle$ and go to Step 1.

Once an ordered pair is removed from $\mathcal{S}$ it is never generated again in Step 7 and the total number of ordered pairs is 2^m. Consequently, since $\mathcal{L}_n$ is finite, Algorithm 2.3 terminates in a finite number of steps. When the algorithm stops, x^* is the optimal solution to (2.1). If Step 4 is executed *only* if $\langle F_0, F_1 \rangle = \langle \emptyset, \emptyset \rangle$, Algorithm 2.3 is equivalent to FCPA (except that in case x^* is the incidence vector of subtours only some subtour elimination constraints are added and the entire enumeration phase is reiterated, which is wasteful).

To describe the branch-and-cut algorithm in detail we use the terminology that is commonly used for branch-and-bound. A *node* associated with the ordered pair $\langle F_0, F_1 \rangle$ is the problem $P(\mathcal{L}_n, F_0, F_1)$, i.e., the problem of finding the minimum length Hamiltonian tour whose incidence vector satisfies the equations

$$
\begin{aligned}
& x_e = 0 \quad \text{for all } e \in F_0, \\
& x_e = 1 \quad \text{for all } e \in F_1.
\end{aligned} \tag{2.5}
$$

We record a node by the ordered pair corresponding to it. A node is *fathomed* if for the optimal solution $\overline{x}$ of the current relaxation $P(\mathcal{L}, F_0, F_1)$ we have $c\overline{x} \geq cx^*$ (Step 3). A node is *solved* if the optimal solution of its current relaxation $P(\mathcal{L}, F_0, F_1)$ is integer. If the optimal solution $\overline{x}$ of the current relaxation $P(\mathcal{L}, F_0, F_1)$ is fractional and no inequalities are found in Step 4, the algorithm picks a *branching variable* and *branches* in Step 7, thus producing two new nodes $\langle F_0 + \{e\}, F_1\rangle$ and $\langle F_0, F_1 + \{e\}\rangle$. The node $\langle F_0, F_1\rangle$ is the *parent* of the two new nodes which are the *children* of $\langle F_0, F_1\rangle$. The nodes $\langle F_0 + \{e\}, F_1\rangle$ and $\langle F_0, F_1 + \{e\}\rangle$ are also called the *down branch* and the *up branch* of $\langle F_0, F_1\rangle$, respectively. The binary tree having a vertex for each node generated in Algorithm 2.3 and an edge for every pair of nodes that are in parent-child relation is called *search tree*. The root of the search tree is the node $\langle\emptyset, \emptyset\rangle$. The nodes of the search tree that are in $\mathcal{S}$ are called *active*.

2.4. Traditional versus polyhedral cutting planes. The linear program $P(\mathcal{L}, F_0, F_1)$ solved at Step 2 of Algorithm 2.3 is a relaxation of the problem $P(\mathcal{L}_n, F_0, F_1)$. Its feasible set is the polytope that is obtained by intersecting Q^n with the hyperplanes defined by the equations (2.5). However, $P(\mathcal{L}, F_0, F_1)$ is a *special* relaxation of $P(\mathcal{L}_n, F_0, F_1)$ because every inequality of $\mathcal{L}$ is not only valid but facet-inducing for the *whole polytope* Q^n. This fact allows all nodes of the search tree to share the same constraint set $\mathcal{L}$, which has enormous advantages in terms of implementation and memory allocation over the traditional cutting-plane/branch-and-bound approach. Moreover, while a node is solved the linear program relaxation related to all other nodes is improved at the same time.

Suppose that a different relaxation of $P(\mathcal{L}_n, F_0, F_1)$ is used where some inequalities are not valid for the whole polytope Q^n. This is the case if classical cutting planes such as, e.g., Gomory cuts (see, inter alia, the book by Garfinkel and Nemhauser (1972)) are used in Step 4. In fact, if $P(\mathcal{G}, F_0, F_1)$, say, is the current relaxation and $\overline{x}$ its optimal solution, a Gomory cut is derived from a suitable linear combination of the inequalities in $\mathcal{G}$ *and* of the equations (2.5) to cut off $\overline{x}$. The equations (2.5), however, are *not* valid for the whole polytope Q^n and consequently the resulting Gomory cut need not be valid for Q^n. Using cuts which are not valid for Q^n makes it necessary to treat and store separately the cuts related to each node of the search tree. This requires additional bookkeeping and a much larger amount of memory. Thus a "marriage" of classical cutting planes and tree-search is out of the question as far as the solution of *large-scale* combinatorial optimization problems is concerned. The only alternative is to avoid branching and to rely solely on (classical) cutting planes. The literature has documented at least one such approach to the STSP. In Miliotis (1978) the integer program (2.3) is solved by way of Gomory cuts only for moderately sized STSPs. Some of the implementational disadvantages of this approach to large-scale STSPs—such as explosive storage requirements—are very clearly spelled out in the conclusions of that paper. Miliotis, however, completely misses the main point, which speaks against the use of *general* cutting planes such as Gomory cuts: It has been known empirically for a rather long while (see Story and Wagner (1963), Wagner, Giglio, and Glaser (1964), Trauth and Woolsey (1969)—and the pertaining "folklore" of the 1960s) that these cutting planes have poor *convergence* properties. Mathematically these observations are explained by the fact that—while *finite* convergence can be proven—classical cutting planes furnish "weak" cuts that frequently do not even define supporting hyperplanes—and certainly not *facets*—of the underlying polytopes (see Padberg (1970), (1971), (1973) for a related discussion of this point in the context of the set packing problem). Realistically, we must at present resort to some

sort of tree-search in the solution of most "hard" combinatorial problems. So there is no choice but to use facet-defining inequalities or polyhedral cutting planes which—as a bonus to their proven mathematical properties—lend themselves *naturally* to a tree-search approach because they are valid *globally*, i.e., across the entire search tree.

The performance of Algorithm 2.3 may change substantially depending on the way the various parts of it are implemented. In particular, we want to mention a few points that are crucial for the algorithm's efficiency and that we are going to address in the following sections:

(a) The effectiveness of fathoming (Step 3) strongly depends on the goodness of the starting solution x^*. In fact the lower the value of the upper bound cx^*, i.e., of the length of the best known tour, the smaller is the number of nodes in the search tree. Therefore it is advisable to execute an efficient heuristic procedure before Algorithm 2.3.

(b) The better the cut generator in Step 4, the higher is the value of $c\bar{x}$ every time Step 2 is executed. Consequently the number of nodes of the search tree may go down drastically if the cut generator is improved. Evidently, the more "efficient" the cut generator the "faster" is the entire phase of constraint generation.

(c) Step 2 has to be executed many times before a solution to (2.1) is obtained. For large-scale instances of STSP a substantial portion of the execution time of the whole algorithm is spent in the linear program optimizer (see § 7). Therefore it is very important to use the LP optimizer as efficiently as possible.

(d) The behavior of the algorithm may change substantially depending on the way the pair $\langle F_0, F_1 \rangle$ in Step 1 and the branching variable in Step 7 are chosen.

3. The heuristic. To find an initial good feasible solution to Problem 2.1 we use a heuristic algorithm. The algorithm is based on the "exchange" heuristic by Lin and Kernighan (1973) which we adapted to handle large instances of the problem. It is a substantial revision of the program used by Padberg and Hong (1980), the original (1975) version of which was a FORTRAN IV program due to Shen Lin. In order to run this heuristic algorithm for problems having several hundred cities and more, we create, in particular, for every node u a list of the MDEPTH closest neighbors, where MDEPTH is a parameter whose default value is 20. For every node in the list we store the node name and the distance from u. The nodes in the list are sorted in ascending order of their distances from u.

The heuristic procedure is run NTRY times and thus finds in general NTRY different "locally best" tours, where NTRY is an input parameter. The starting tour for the heuristic is constructed in two different ways. If the counter of the trials of the heuristic is odd the starting tour for the exchange heuristic is a "nearest neighbor" tour which is constructed from a randomly chosen node. If the counter is even the starting tour is determined randomly.

In both cases we used the random number generator described in Lewis and Payne (1973) initialized with the parameters $p = 98$, $q = 27$, $delay = 71$ and implemented for 16-bit numbers. The generator produces pseudorandom numbers uniformly distributed in the interval $[0, 1]$.

The best among the NTRY tours generated by the heuristic is taken as current best tour and the union of the edges of the NTRY tours is passed to the branch-and-cut algorithm for the initialization of the "active" set of variables (see § 5.1.) As a default value for NTRY we recommend $\min\{50, \max\{10, 0.1n\}\}$.

4. The constraint generator. The constraint generator is the most important component of our polyhedral cutting-plane algorithm. The generated inequalities are

all facet-inducing for Q^n and fall into one of the following four categories:
(a) Subtour elimination inequalities,
(b) 2-Matching inequalities,
(c) Comb inequalities,
(d) Clique-tree inequalities.
The inequalities (a), (b) and (c) have been proven to induce facets of Q^n in Grötschel and Padberg (1979a), (1979b) and inequalities (d) in Grötschel and Pulleyblank (1986). For definitions and for a complete survey see Grötschel and Padberg (1985). All four types of constraints have the general form

$$\sum_{q=1}^{h} x(E(H_q)) + \sum_{p=1}^{t} x(E(T_p)) \leq \sum_{q=1}^{h} |H_q| + \sum_{p=1}^{t} (|T_p| - t_p) - \frac{t+1}{2},$$

where $H_q \subseteq V$ for $q = 1, \cdots, h$ and $T_p \subseteq V$ for $p = 1, \cdots, t$, respectively, are disjoint sets called *handles* and *teeth*, respectively. These subsets satisfy:
(1) $2 \leq |T_p| \leq n-2$ and $|T_p - \bigcup H_q| \geq 1$ for each p.
(2) Each handle intersects an odd number (≥ 3) of teeth.
(3) t_p is the number of handles that tooth T_p intersects; the total number of teeth t is odd.
(4) The intersection graph of the system of sets given by the handles and teeth forms a tree.
If $h = 1$ and $t = 0$ we have a subtour elimination constraint (the right-hand side is replaced by $|H| - 1$); if $h = 1$ and $t \geq 3$ we have a 2-matching constraint ($|T_p| = 2$ for all p) or a comb constraint, while if $h \geq 2$ and $t \geq 5$ we have a general clique-tree constraint. For the subtour elimination inequalities Problem 2.2 can be solved in polynomial time by the Gomory–Hu algorithm for the determination of a minimum weighted cut in an undirected graph (see Crowder and Padberg (1980)). Padberg and Rao (1982) give a polynomial algorithm to solve Problem 2.2 for 2-matching inequalities that is based on the Gomory–Hu algorithm. At present, no polynomial algorithm is known for the other families of inequalities. In Padberg and Rinaldi (1990a) we give several heuristic procedures for the identification of comb inequalities and some special clique-tree inequalities that we call *basic clique-trees*. We also describe an exact identification procedure for subtour elimination inequalities that uses the minimum-cut algorithm of Padberg and Rinaldi (1990b) and an exact Padberg–Rao procedure for the identification of 2-matching inequalities that has a time complexity of $O(n^4)$. The latter being very time consuming for large-scale TSP instances, we also propose three different heuristic procedures for the identification of these inequalities.

The cut generator that we use in our algorithm is described in detail in Padberg and Rinaldi (1990a). To avoid duplication we do not repeat here the various algorithms of that paper. Rather we refer to them by the numbering used there and mark them with an asterisk to avoid confusion with the numbering used in this paper.

Our cut generator is an implementation of Algorithm 8.1* where all but one of the above-mentioned procedures are used. We never use the exact procedure for the identification of 2-matching constraints because the Gomory–Hu algorithm is too expensive in terms of time and memory allocation. Therefore we use a first quick heuristic (Procedure 5.3* at Step 5*) and then a slower but more efficient heuristic (Procedure 4.11* at Step 8*). Procedure 4.11* requires repeated executions of a max-flow algorithm, and hence Step 8*, which is much more time consuming than Step 5*, is executed only if a flag DOHARD is set equal to *true*.

The cut generator runs Algorithm 8.1* a first time with the flag DOHARD set to *false*. If the number of cut generated is less than 5 and $n \leq 120$, or if it is less than 10 and $n > 120$, then Algorithm 8.1* is executed again with the flag DOHARD set to *true* while skipping Step 1*, where the subtour elimination constraints are generated. Moreover, the flag DOHARD is set to *true* the first iteration after the active set of variables has been augmented (see § 5.1) or if two successive objective function values fall below the value GIVEUP (see § 4.3).

Given a solution $\overline{x}$ to the current LP relaxation the *slack* of an inequality is the value of its right-hand side minus the left-hand side evaluated at $\overline{x}$. We declare a constraint violated if its slack is negative and has an absolute value bigger than a parameter XTOLCT. Assume that the LP solver guarantees a *precision* of p decimal digits in the floating point representation of each component of $\overline{x}$. To be conservative the parameter XTOLCT is set to a value greater than $2q10^{-p}$, where q is the size of the current basis.

4.1. Data structure for the cut generator. The input to the cut generator is the optimal solution $\overline{x}$ of the current LP relaxation. We represent $\overline{x}$ by its *support graph*, i.e., the weighted graph $(G, \overline{x}) = (V, F, \overline{x})$, where F is the subset of E corresponding to all nonzero components of $\overline{x}$. The weight of each edge $e \in F$ is $\overline{x}_e$.

We represent $(G, \overline{x})$ by a data structure that is used commonly for sparse graphs. The nodes of V are indexed with the integers $\{1, \cdots, |V|\}$, the edges of F are indexed with the integers $\{1, \cdots, |F|\}$. For every node $u \in V$ let $\delta(u)$ denote the set of edges of F incident with u. We give an arbitrary order to the elements of $\delta(u)$ so that the *first* and the *last* elements of $\delta(u)$ and the successor and the predecessor of an element of $\delta(u)$ are well defined. The predecessor of the first and the successor of the last element is the *nil* element. We store $\delta(u)$ with a doubly linked list, i.e., every element of the list has a pointer to its successor and a pointer to its predecessor. This way the operations of insertion and deletion of an element in the list representing $\delta(u)$ take constant time.

To store the data structure we use seven arrays FR, TO, W, NXT1, NXT2, NXT3, and NXT4 dimensioned to $|F|$ and an array HEAD dimensioned to $|V|$. Let i be the index of the edge $e = (u, v) \in F$, j the index of node u, k the index of node v, and $j < k$, then:

- FR(i) contains j,
- TO(i) contains k,
- W(i) contains $\overline{x}_e$,
- NXT1(i) contains the index of the successor of e in $\delta(u)$,
- NXT2(i) contains the index of the successor of e in $\delta(v)$,
- NXT3(i) contains the index of the predecessor of e in $\delta(u)$,
- NXT4(i) contains the index of the predecessor of e in $\delta(v)$.

As is customary the index zero represents the nil element. Let j be the index of a node $v \in V$. Then HEAD(j) contains the index of the first element of $\delta(u)$.

This data structure permits us to efficiently carry out the operations of the identification procedures of Algorithm 8.1*. In particular it is designed to quickly perform the *shrinking* operation of two nodes into a single node. This operation is used very often in order to reduce the size of the graph that is the input to an identification procedure and is essential in the identification of *clique-liftable* inequalities (see Padberg and Rinaldi (1990a)). The procedure that performs shrinking with the above data structure is described in detail in Padberg and Rinaldi (1990b). In particular, with every node of V we associate a "cluster" of nodes. The clusters are represented

by simply linked lists stored in the array CLUST. Initially the cluster of u contains only u. Given two nodes u and v with index j and k, respectively, and $j < k$, the list representing the cluster of v is merged to the list of u, and u is removed from the data structure if u and v are shrunk.

For some identification routines we have to represent the *fractional graph*, which is obtained from $(G, \overline{x})$ by removing its 1-edges, i.e., the edges with weight 1. To keep track of the removed edges we make use of an additional array PARITY, dimensioned to $|V|$. If j is the index of a node v, PARITY(j) indicates whether v is an isolated node (two 1-edges are incident with it), an *odd* node (only one 1-edge is incident with it), or an *even* node (no 1-edge is incident with it). If j and k are the index of the nodes at the extremities of a chain of 1-edges, PARITY(j) contains k and PARITY(k) contains j.

4.2. Constraint reduction. The *support* of an inequality is the partial subgraph of K_n that is spanned by the edges having nonzero coefficients in the inequality. During our computational experiments we observed that—possibly as a result of our shrinking procedures—many of the inequalities found by the cut generator had "large" support. A constraint with large support is dense and contributes to the increase of the fill-up of the constraint matrix and of the basis. As discussed in § 5, keeping the constraint matrix sparse is one of the ways to increase the efficiency of the LP solver. Therefore we developed procedures to reduce the size of the support graph. To do so we use the slack value as a measure of the quality of a cut: The more negative the slack the better the cut (see § 5.3). Consequently, we try to keep the slack value as negative as possible in the reduction procedure and we reject a possible constraint reduction if the absolute value of the slack of the reduced constraint is too small.

The cut generator produces the subtour elimination inequalities in the following form

$$x(E(W)) \leq |W| - 1, \tag{4.1}$$

where W is a subset of V with $3 \leq |W| \leq |V| - 3$. The inequality (4.1) is equivalent to the one obtained by replacing W by $V - W$, i.e., the two inequalities define the same facet of Q^n. We exploit this equivalence by replacing W with its complement $V - W$ in (4.1) whenever $|W| \geq n/2$.

For any subset $W \subseteq V$ let G^W be the subgraph of G induced by W. If the subgraph G^W induced by a subtour elimination constraint is connected but not biconnected we decompose it into its blocks. Let $W_1, \cdots, W_k$ be the node sets of the blocks of G^W. Then we have

$$\overline{x}(E(W)) = \sum_{i=1}^{k} \overline{x}(E(W_i)).$$

Therefore at least one of the inequalities $x(E(W_i)) \leq |W_i| - 1$, $i = 1, \cdots, k$, has slack less than or equal to the slack of (4.1). On the other hand, some of these inequalities may not be violated at all. Among these k inequalities we generate all those having a slack of less than the tolerance $-$XTOLCT.

We apply several reduction operations to the comb and clique-tree inequalities. The reduction procedure is applied to each comb inequality as well as to each *building comb* (see Padberg and Rinaldi (1990a)) of a clique-tree inequality.

For any comb inequality let H be its handle, $\mathcal{T} = \{T_i, i = 1, \cdots, t\}$ the set of its teeth, and s its slack. For every tooth T that intersects exactly one handle H of a comb or a clique-tree we perform the following three operations.

REDUCTION A. Let Z be the maximal subset of $H \cap T$ such that $N(v) \cap (H \backslash T) = \emptyset$ for each $v \in Z$, where $N(v)$ is the set of nodes of G adjacent to v. If $Z = H \cap T$ we do not reduce. Otherwise we set $\overline{H} = H - Z$ and compute the slack $\overline{s}$ of the comb obtained by replacing H with $\overline{H}$. If one of the following two conditions

(a) the constraint to be reduced is a comb and $\overline{s} < s/2$,

(b) the constraint to be reduced is a building comb and $\overline{s} < s + \text{XTOLCT}$,

is satisfied we replace H with $\overline{H}$ and set $s = \overline{s}$ (see Fig. 2, where the nodes of Z are shown black).

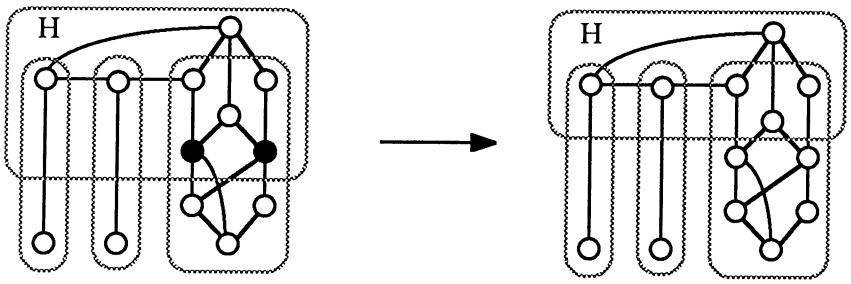


FIG. 2. *Reduction* A.

REDUCTION B. Partition T into three subsets Y, W and U, where $Y = H \cap T$, $W = \{u \in T \backslash H \mid N(u) \cap Y \neq \emptyset\}$ and $U = T - Y - W$. Let $\overline{s}$ be the slack of the inequality obtained by replacing T with $Y \cup W$. If $\overline{s} < s + \text{XTOLCT}$, we set $s = \overline{s}$ and $T = Y \cup W$ (see Fig. 3).

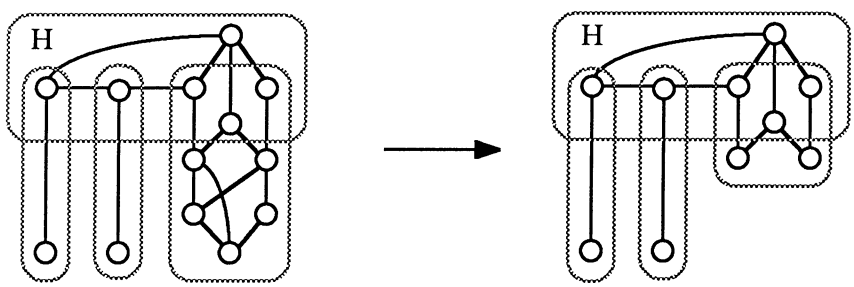


FIG. 3. *Reduction* B.

REDUCTION C. If there is a 1-edge with both its endpoints in $T \backslash H$ and one of its endpoints, say v, has degree 1 in the graph G^T, then we set $T = T - \{v\}$. This reduction operation is repeated until there are no more 1-edges satisfying the condition (see Fig. 4, where 1-edges are represented by double lines).

Reduction C permits us to remove a long path of 1-edges from a tooth.

The following three reductions concern the handle H. Let H_0 denote the set $H \backslash \bigcup_{i=1}^{t} T_i$.

REDUCTION D. If there is a path $[u_1, u_2, \cdots, u_h]$ of 1-edges in G^{H_0} and $h \geq 4$, we replace H by $H - \{u_2, \cdots, u_{h-1}\}$, H_0 by $H_0 - \{u_2, \cdots, u_{h-1}\}$, t by $t+2$, and add to $\mathcal{T}$ the two new teeth $\{u_1, u_2\}$ and $\{u_{h-1}, u_h\}$ (see Fig. 5, where 1-edges are represented by double lines).

REDUCTION E. Suppose that the graph G^H contains two adjacent edges $[u, v]$ and $[v, w]$ satisfying the following conditions:

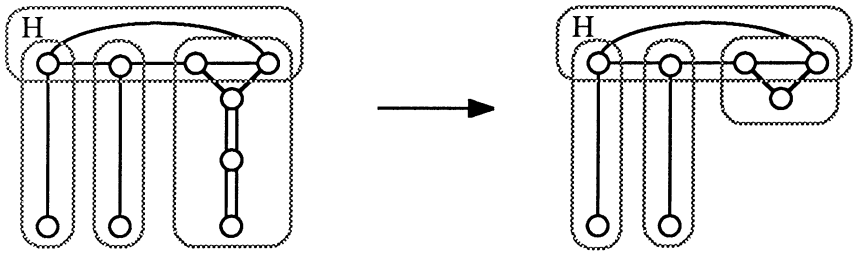


FIG. 4. *Reduction* C.

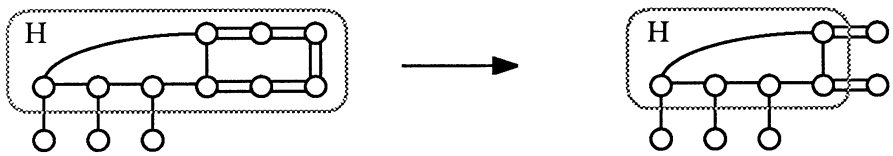


FIG. 5. *Reduction* D.

(a) w has degree 1,
(b) $\overline{x}(u,v) = 1 - \overline{x}(v,w)$,
(c) $\{v,v'\}$ and $\{w,w'\}$ belong to $\mathcal{T}$ and $\overline{x}(v,v') = \overline{x}(w,w') = 1$.
In this case we replace $\mathcal{T}$ by $\mathcal{T} - \{v,v'\} - \{w,w'\}$, H by $H - \{v,w\}$, and t by $t-2$. We repeat Reduction E until there are no more pairs of adjacent edges satisfying the conditions (a), (b), and (c) (see Fig. 6, where 1-edges are represented by double lines and the other edges are labelled with their corresponding coefficients in $\overline{x}$).

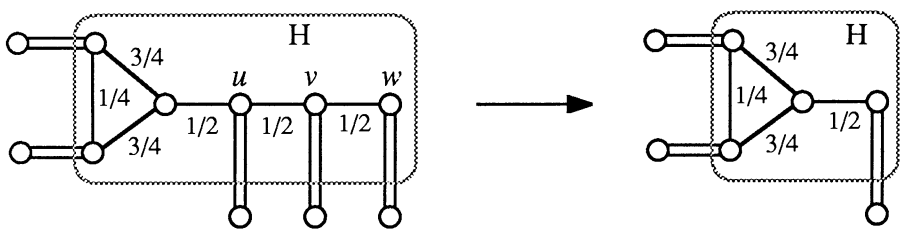


FIG. 6. *Reduction* E.

REDUCTION F. If the constraint to be reduced is a comb and $|H| \geq n/2$ we replace H by $V - H$ since the resulting inequality is equivalent to the original one. Then we apply to the latter inequality all of the reductions A,$\cdots$,E.

4.3. Tailing-off. Table 2 shows some results of a particular run of our algorithm. The second column of the table gives the optimal objective function value ZLP for the node $\langle \emptyset, \emptyset \rangle$ of Problem GIL249 (see § 7) at different iterations of Algorithm 2.3. The iteration counter is incremented every time constraints are added to the linear program (we are here not counting the iterations due to an increase of the "active" variables set; see § 5.1) and the respective objective function values ZLP are reported in the second column of the table. The third column gives the CPU time in seconds of a VAX 780 spent from the beginning of the algorithm. This time includes 2974

seconds spent to run the heuristic 50 times and 107 seconds to get an optimal solution over Q_A^n which has an objective function value of 2221.5.

TABLE 2

Iteration	LP value	Time
1	2294.500	3153
2	2324.000	3187
3	2338.500	3302
4	2356.000	3333
5	2360.500	3352
10	2367.181	3523
15	2369.807	3696
20	2371.782	3881
25	2373.500	4137
32	2375.000	4589
40	2376.000	5497
53	2376.300	6751
88	2376.402	12663
111	2376.439	19304

The value ZLP increases quickly in the first 10 iterations, then it increases by less than 9 units in the following 30 iterations and finally it takes 71 iterations to increase by less than half a unit. This "saturation" of ZLP is called *tailing-off* and reveals the inability of the cut generator to produce the "right" cut that would take the current optimal LP solution out of the corner of the polytope where it is "trapped." The stronger the cut generator, the higher is the value of ZLP where the tailing-off occurs. In fact, while we were developing the cut generator we observed that this value increased every time we added a new identification procedure.

To give some substance to our point of view that tailing-off is not inherent to the use of cutting planes but rather a question of generating the "right" cut, we report here the (previously unpublished) results of an experiment carried out by Rinaldi and Yarrow (1985): The 48-city problem ATT048 (see § 7) of the 48 capitals of the continental United States was solved to optimality. The (unique) optimal solution was found without branching by a man-machine polyhedral cutting-plane procedure in the vein of the work of Dantzig, Fulkerson, and Johnson (1954), i.e., the constraint identification was performed "visually" and the identified constraints were added "by hand" to the LP relaxation. The LP optimization was done using the LINDO package. The exercise was done three times and the values of the objective function at every iteration are shown in Table 3 and plotted in Fig. 7. In the first run the optimal solution was found after 21 iterations. Then the whole procedure was done a second time and the "learning" that had resulted from the first run can be seen quite strikingly. This run ended in 12 iterations. Finally, to simulate the behavior of an automatic "perfect" facet-identification procedure a "pool" was created from all the inequalities generated in the second run and a procedure was written to generate cuts from the pool as soon as they were violated by the current LP solution. Only five iterations were necessary this time to find the optimal solution. No tailing-off can be inferred from the third run, while the first two runs clearly exhibit this phenomenon due to the "incompleteness" of the (human) constraint identification. As most of our constraint identification procedures are *heuristic* and a reasonable measure of what constitutes the right cut is missing, behavior similar to that observed in runs I and II must rea-

sonably be expected, even when our "automatic" constraint generation procedures are used. The type of experiment described here has since been repeated (eight times) in the dissertation of Araque (1989) and always confirmed the above conclusions.

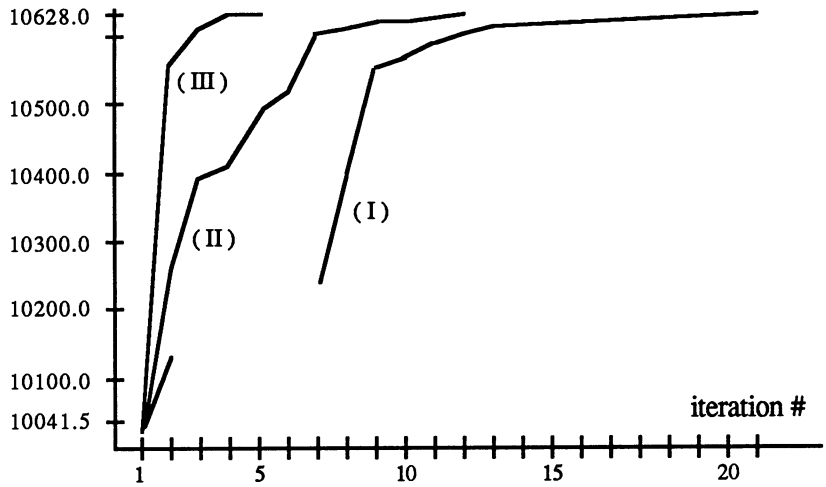


FIG. 7. *Three runs of* ATT048.

TABLE 3

Iteration #	First run	Second run	Third run
1	10041.5	10041.5	10041.5
2	10124.3	10269.0	10563.2
3	— (*)	10360.5	10617.0
4	— (*)	10410.5	10627.0
5	— (*)	10487.0	10628.0
6	— (*)	10517.8	
7	10241.0	10602.0	
8	10401.0	10614.7	
9	10558.0	10617.0	
10	10571.5	10623.0	
11	10592.5	10624.0	
12	10604.0	10628.0	
13	10610.3		
14	10611.0		
15	10617.0		
16	10618.0		
17	10623.0		
18	10623.0		
19	10624.0		
20	10626.8		
21	10628.0		

(*) data lost

Obviously, when tailing-off is recognized it is probably worthwhile to stop the cut generation and to resort to branching. Unfortunately, it is difficult to recognize the

right point at which to give up. If the cut generation on a node is stopped too early the running time of the entire algorithm may increase, since the size of the search tree increases. We adopted the following simple strategy to control the tailing-off: If for l iterations in a row the increase of ZLP stays below a certain parameter GIVEUP, then the cut generation is stopped. In the current implementation of the algorithm l is set to 6 unless we are *reconstructing* a parent node (see § 6.2) in which case it is set equal to 2. The default value for GIVEUP is $2.5 \times 10^{-5} \times$ ZLP.

Problem GIL249 is an extreme case of the tailing-off phenomenon. In the other problems of our computational study the sequence of small increases of ZLP terminates much earlier because the cut generator finds no more cuts. GIL249 was solved in a total time of 14,037 seconds when our give-up strategy was implemented with the above default values of the parameters.

5. The interface with the LP optimizer. Since we are solving TSP's on complete graphs the number of variables of every linear programming problem $P(\mathcal{L}, F_0, F_1)$ is rather large (order of n^2). The number of constraints of the largest linear relaxation depends on the problem instance and on the efficiency of the cut generator. In our computational experience it is about $2n$. The first n constraints are the rows of the matrix A with a density of nonzero entries of $2/n$ since there are two nonzero entries per column. The remaining constraints are the inequalities of $\mathcal{L}$. The density of this part of the constraint set is in general higher than $2/n$. Therefore even though techniques for sparse matrices are used, storing the entire constraint matrix for a problem with, e.g., 1000 or more nodes explicitly would not be practical or even possible with the present computer technology.

The time taken by the LP solver to solve a problem strongly depends on the size and the density of the basis. Moreover, the time taken by the pricing operation depends on the number of variables.

There are a few observations that permit us to overcome these problems. The set of cuts $\mathcal{L}$ grows every time Step 5 of Algorithm 2.3 is executed. However, not all of the inequalities in $\mathcal{L}$ are binding at the optimal solution of the current linear program $P(\mathcal{L}, F_0, F_1)$. In our experience the number of cuts which are binding is on the average about $n/2$ and hardly ever goes beyond n. Therefore the number of variables that are basic at the optimal solution is at most $2n$. In addition because of the degree equalities, at most n variables can be nonbasic at their upper bounds. Consequently, only about $3n$ out of the total of $m = n(n-1)/2$ variables are nonzero at the optimal solution.

5.1. Column generation and deletion. The observations of the previous paragraph can be exploited to solve $P(\mathcal{L}, F_0, F_1)$ in Step 2 of Algorithm 2.3 by solving one or possibly more linear programs having a smaller number of variables and constraints than $P(\mathcal{L}, F_0, F_1)$. We now give a procedure that outlines how we obtain a substantial reduction of the number of variables.

Given a set $J \subset E$, a vector $x \in \mathbb{R}^E$, and a matrix B whose columns are indexed by the elements of E, we denote by x_J the projection of x into the subspace $\mathbb{R}^J$ and by B_J the submatrix of B consisting of the columns with index in J. For $e \in E$ we denote by $B_{\{e\}}$ the column of B with index e. Given a set of linear inequalities $\mathcal{L} \subset \mathcal{L}_n$, we denote by $Lx \leq l$ its matrix representation, where L has size $|\mathcal{L}| \times m$.

PROCEDURE 5.1. *Input:* $P(\mathcal{L}, F_0, F_1)$.

Step 1. Select a subset $J \subset E$ with $|J| \ll |E|$ such that the partial graph $G = (V, J)$ is Hamiltonian and $F_0 \subseteq J$, $F_1 \subseteq J$. We call J the set of *active* variables of $P(\mathcal{L}, F_0, F_1)$.

Step 2. Solve the linear program

$$\begin{aligned} \min \quad & c_J x_J \\ \text{s.t.} \quad & A_J x_J = \mathbf{2}, \\ & L_J x_J \leq l, \\ & x_e = 0 \quad \text{for all } e \in F_0, \\ & x_e = 1 \quad \text{for all } e \in F_1, \\ & \mathbf{0} \leq x_J \leq \mathbf{1}, \end{aligned}$$

and let $\overline{x}_J$ be the optimal solution and $\overline{u}$ and $\overline{v}$ the optimal dual vectors corresponding to the degree equations and the inequalities, respectively.

Step 3. Compute the reduced cost

$$r_e = c_e - \overline{u}A_{\{e\}} - \overline{v}L_{\{e\}} \quad \text{for all } e \in E - J.$$

If $r_e \geq 0$, for all $e \in E - J$, stop.

Step 4. Replace J by $J + \{e \in E - J \mid r_e < 0\}$ and go to Step 2.

When Procedure 5.1 stops with a feasible solution, then the vector $\overline{x} \in \mathbb{R}^E$ obtained from $\overline{x}_J$ by setting $\overline{x}_e = 0$ for all $e \in E - J$ is an optimal solution of $P(\mathcal{L}, F_0, F_1)$. The case when the linear program in Step 2 does not have a feasible solution is also possible and will be dealt with in § 6.5. Even though the cardinality of J never decreases during the procedure as described above, it is usually much smaller than $|E|$, and storing the matrix L_J requires a substantially smaller amount of memory. Moreover, we *delete* columns in J as well whenever this is mathematically correct (see § 5.5).

Procedure 5.1 is used in Step 2 of Algorithm 2.3. Initially the set J of active variables is the union of the NTRY locally best tours found by the heuristic. If branching occurs in Algorithm 2.3 the initial set of variables in the "next" problem is simply the set of active variables prior to branching.

Since Step 3 of Procedure 5.1 is rather time consuming we do not execute it every time a linear program is solved. More precisely, Steps 3 and 4 are executed only if one of the following situations occurs:

(a) $\mathcal{L}$ is empty.
(b) The number of times the LP solver has been executed is a multiple of 5.
(c) The current problem has an integer solution or no inequalities are generated that cut off the current fractional solution.
(d) The conditions to fix variables (see § 5.5) are satisfied.

It is conceivable that the set J may grow too much if Steps 3 and 4 are executed too *infrequently*. We found a frequency of one execution every five LP computations to be a good compromise. To speed up the calculation of the reduced cost we exploit the fact that the constraint matrix and the vector $\overline{v}$ are nonnegative. Therefore in Step 3 of Procedure 5.1 we initialize r_e to be $c_e - \overline{u}A_{\{e\}}$ and we stop the computation of the inner product $\overline{v}L_{\{e\}}$ as soon as the partial value of r_e becomes negative. We also use this *stopped* calculation of the reduced cost to check if a variable can be fixed or set (see § 5.5).

We give a few additional details that facilitate the implementation of our column generation scheme. The nodes of V are indexed with the integers $1, \cdots, n$. The edges of E are indexed with the integers $1, \cdots, |E|$ in the natural way. That is, if i_h and j_h $(i_h < j_h)$ are the indices of the endpoints of the edge indexed by h and i_k and j_k

$(i_k < j_k)$ are those of the edge indexed by k, then $h < k$ implies that either $i_h < i_k$ or $i_h = i_k$ and $j_h < j_k$. Consequently, the indices of the endpoints of the edge indexed by k are given by

$$(5.1) \quad i_k = n - \left\lfloor \frac{1 + \sqrt{1 + 4n(n-1) - 8k}}{2} \right\rfloor, \qquad j_k = i_k + k - \frac{(2n - i_k)(i_k - 1)}{2},$$

where $\lfloor z \rfloor$ denotes the largest integer less than or equal to z. The edges of J do not have any special order and are indexed with the integers $1, \cdots, \text{NVAR}$. For every $1 \leq e \leq \text{NVAR}$ the eth entries of the two arrays N0FR and N0TO contain the endpoints of edge e, with $\text{FR}(e) < \text{TO}(e)$. The nonactive variables are not explicitly stored and Steps 3 and 4 are executed by varying k from 1 to m and generating the column associated with the edge $[i_k, j_k]$, where i_k and j_k are obtained by (5.1). Before adding a variable to J we check whether it is already contained in J to avoid duplication. Since the added variable has a negative reduced cost only the variables of J that are nonbasic at their upper bound need to be checked.

5.2. Constraint storage. Because of the very large number of variables it is out of the question to store all nonzero entries of every inequality. Furthermore, because the set J of active variables changes *dynamically*, we cannot restrict the set of variables to this subset either. Due to the combinatorial description of the inequalities in $\mathcal{L}$ there is, however, a way to store these constraints compactly.

We exploit the fact that the left-hand side of the constraints produced by the cut generator has the form

$$(5.2) \qquad \sum_{i=1}^{h} x(E(H_i)) + \sum_{j=1}^{t} x(E(T_j)).$$

The sets $H_i \subset V$, $i = 1, \cdots, h$ are pairwise disjoint and so are the sets $T_j \subset V$, $j = 1, \cdots, t$. Consequently any node $v \in V$ belongs to at most two of the sets that appear in (5.2). For subtour elimination inequalities $h = 1$ and $t = 0$; for 2-matching and comb inequalities $h = 1$ and $t \geq 3$ and for clique-tree inequalities $h \geq 2$ and $t \geq 5$. Since every set T_j has cardinality at least 2, t cannot exceed $n/2$.

To store the cut $lx \leq l_o$ having a left hand side (5.2) we use a vector y of length $|V|$ defined as follows:

$$y_v = \begin{cases} 0 & \text{if } v \notin \bigcup T_j \cup \bigcup H_i, \\ p & \text{if } v \in T_p,\ v \notin \bigcup H_i, \\ (\lfloor \frac{n}{2} \rfloor + 1)q & \text{if } v \in H_q,\ v \notin \bigcup T_j, \\ (\lfloor \frac{n}{2} \rfloor + 1)q + p & \text{if } v \in H_q \cap T_p\ . \end{cases}$$

Given an edge $e = [u, v]$, the corresponding coefficient of l is given by

$$(5.3) \qquad l_e = \begin{cases} 2 & \text{if } y_u = y_v > \lfloor \frac{n}{2} \rfloor + 1 \text{ and } \hat{y}_u \neq 0, \\ 1 & \text{if } \hat{y}_u = \hat{y}_v \neq 0 \text{ and } y_u \neq y_v, \\ 1 & \text{if } 0 < y_u = y_v < \lfloor \frac{n}{2} \rfloor + 1, \\ 1 & \text{if } 0 < y_u = y_v \text{ and } \hat{y}_u = 0, \\ 0 & \text{in all other cases,} \end{cases}$$

where by $\hat{y}_w$ we denote $y_w \bmod (\lfloor \frac{n}{2} \rfloor + 1)$. This way only n memory locations are necessary to store the left-hand side of a constraint and every column of the matrix L can be generated quickly using (5.3).

5.3. Row generation and deletion. As already observed, not all of the inequalities of $\mathcal{L}$ are satisfied at equality by an optimal solution $\overline{x}$ of a linear program $P(\mathcal{L}, F_0, F_1)$. We call all inequalities whose corresponding slack is nonbasic *active* and we denote by $\mathcal{L}_a \subseteq \mathcal{L}$ the set of active inequalities. The inequalities of the set $\mathcal{L}_p = \mathcal{L} - \mathcal{L}_a$ are called *nonactive*. The problem $P(\mathcal{L}_a, F_0, F_1)$ has the same optimal solution $\overline{x}$ but typically a much smaller basis. Let $\mathcal{L}'$ be the set of inequalities that are produced by the cut generator. The problem $P(\mathcal{L}_a \cup \mathcal{L}', F_0, F_1)$ can be solved more efficiently than $P(\mathcal{L} \cup \mathcal{L}', F_0, F_1)$ because it has a smaller basis. However, the optimal solution of the first problem may differ from that of the second because it may violate some of the inequalities of $\mathcal{L}_p$. These inequalities are then added to $\mathcal{L}_a$ at the next iteration of Algorithm 2.3.

We call the operation of removing from the constraint set all inequalities with a basic slack *purging*. To avoid cycling we purge constraints only if the objective function value is strictly higher than the value of the previous linear program. We could simply remove a purged constraint from the data structure and then let the cut generator produce it again in case the constraint is violated by some later LP solution. However, since many procedures of the cut generator are not exact it is entirely possible that the cut generator find a violated constraint if x^1 is given as input but not if x^2 is given instead when both x^1 and x^2 violate the same constraint.

Therefore we store the purged constraints in a separate data structure that we call the *pool*. Given a fractional solution to the current linear program, we check whether any of the constraints in the pool are violated before calling the cut generator. If this is the case these constraints are removed from the pool and added to the active set. Some of these inequalities may be produced again by the cut generator, so before adding an inequality identified by the cut generator, we check whether it is already in the active set in order to avoid constraint duplication.

In general the number of cuts in $\mathcal{L}$ increases at every iteration. While the cardinality of $\mathcal{L}_a$ is on the average below n, the cardinality of $\mathcal{L}_p$ increases up to many times n. It may happen that at a certain point the pool is full and so it becomes necessary to delete some inequalities permanently from the pool following some priority rules.

We never store a subtour elimination inequality in the pool, since an exact identification procedure for these inequalities is part of the cut generator. When the processing of a node of the search tree is terminated because no more cuts can be generated, we *mark* all constraints that are currently active. These constraints are all necessary to *reproduce* the best LP relaxation of the node constructed by the algorithm and they should be in the active set when one of the children of the node is chosen as the current node. To do so ensures the monotonicity of the objective function value along each path in the search tree. Though such monotonicity is desirable, it is not necessary for the validity of the algorithm.

The unmarked constraints that have a slack value (computed at the current LP solution) higher than a fixed parameter are the first to be deleted. Then we delete the other unmarked constraints and only if the pool is made up totally of marked constraints are some of them deleted. In any case, a marked constraint is never deleted to make room for a purged constraint that is not marked.

The pool is particularly efficient in the context of branch-and-cut for the following reason. Suppose that k runs of the LP solver are necessary to obtain the relaxation $P(\mathcal{L}, F_0, F_1)$ of the node $\langle F_0, F_1 \rangle$, starting from $P(\emptyset, F_0, F_1)$ and using the cut generator of § 4, and that we put the inequalities of $\mathcal{L}$ into the pool. If we start the process again from $P(\emptyset, F_0, F_1)$ but use the cuts in the pool instead of the cut generator, we obtain the optimal solution of $P(\mathcal{L}, F_0, F_1)$ in a number of iterations that is consider-

ably smaller than k, because the inequalities become active earlier on than in the first case (see our experiment mentioned in § 4.3). For reasons explained below (§ 6.2), we *reconstruct* the linear relaxation of a parent node before processing its children and the pool permits us to do the reconstruction in a more efficient way.

An additional device we use to reduce the size of the basis of the linear programs is based on the following observation: Let $\overline{x}$ be the optimal solution of the problem $P(\mathcal{L}, F_0, F_1)$, $\mathcal{L}_a$ be the active set and $\mathcal{L}'$ be the inequalities identified by the cut generator. Usually only a small subset of $\mathcal{L}'$ belongs to the active set of the next linear program $P(\mathcal{L}_a \cup \mathcal{L}', F_0, F_1)$. If we knew this subset in advance we would add only the corresponding inequalities to $\mathcal{L}_a$ and solve the next LP faster. The cuts of this set are in some sense the right cuts produced by the cut generator and could be recognized in advance if a "quality" measure was available for a cut. In our estimation, finding a reasonable quality measure is one of the central issues in the area of polyhedral cutting-plane algorithms that is—as of today—not yet investigated satisfactorily. A possible measure of the quality of a cut is the distance of $\overline{x}$ from the projection of the cut into the affine space defined by the degree equations. The computation of the projection is, however, rather time consuming and so we use as a measure the absolute value of the slack of the constraint evaluated at $\overline{x}$. The larger this value the better the cut, which is the standard concept of a "most-violated" constraint.

We sort the cuts produced by the cut generator in descending order of this measure. The first s cuts are added to the current active set. The remaining cuts are *suspended.* At the next iteration the suspended cuts that are violated by the new LP solution go into the active set. The other suspended cuts go into the pool. In our implementation the parameter s is set to $\max\{10, (2n/3 - |\mathcal{L}_a|)/10 + 1\}$, which came about by our empirical observation that, at the *optimum*, hardly ever are more than $2n/3$ cuts active. Due to the incompleteness of the cut generator the *suspend/resume* strategy tends also to increase the number of cuts produced by the cut generator because it is called for a greater number of points $\overline{x}$. However, when the flag DOHARD (see § 4) is set to *true*, no constraint is suspended; in this case we let the LP solver "select" the active constraints.

The average number of active constraints including the degree equations is smaller than the average number of active variables. For this reason the primal rather than the dual of each linear program is solved at every LP iteration in order to keep the basis as small as possible. This choice makes row addition/row deletion somewhat more complicated since LP solvers store the constraint matrix typically columnwise.

5.4. The LP solver. For the development of the software system we used the XMP package of Marsten (1981) as the linear program solver. XMP is written in FORTRAN and consists of many routines. From among them we use in our implementation of Algorithm 2.3 only the following ones: XMAPS to initialize the internal XMP data structure; XSTART to factorize the basis; XPRIML to execute a primal simplex algorithm; XDUAL to execute a dual simplex algorithm and XADDAJ to add a column to the constraint matrix.

Since some large highly degenerate problems did not terminate with the original code we made the following changes:

(a) We set the refactorization frequency parameter FACTOR to 20.

(b) In the original code the algorithm abandons the solution of a problem after a limit of $3 \times$ FACTOR degenerate pivots. Since our linear programs are highly degenerate we changed the limit to $200 \times$ FACTOR.

(c) We modified the pivot rejection strategy in XDUAL: Every time the chosen pivot

element has a value smaller than a certain parameter value, the selected entering column is rejected and the simplex algorithm looks for another entering column. After 10 rejections in a row the algorithm gives up. Our modification consists in refactorizing the basis after the first rejection. In fact it may happen that a sequence of 10 pivot rejections occurs simply because of an error accumulation due to a numerical inaccuracy. Refactorizing the basis removes the error accumulation and the roundoff errors disappear. In our computational experience all problems were solved by the LP solver without anomalies after we made these changes.

(d) We introduced an additional parameter ZCUTOF to the parameter list of the routine XDUAL. Every time the objective function value exceeds ZCUTOF the routine XDUAL stops. In the branch-and-cut algorithm ZCUTOF is set equal to cx^*. If the objective function value of the current LP exceeds this value, the corresponding node can probably be fathomed and the optimization can be stopped before an optimal solution is obtained. Because we use the dual simplex method this proceeding can be embedded into a mathematically correct procedure for the Algorithm 2.3.

Although the routines of XMP can treat both equations and inequalities, we choose to introduce the slacks ourselves. If the linear program has NVAR structural variables and NCUT inequalities, the indices of the structural variables go from 1 to NVAR and those of the slacks go from NVAR + 1 to NVAR + NCUT. Therefore all constraints of our problems are treated as equations by XMP.

We use both the primal simplex method XPRIML and the dual simplex method XDUAL, but XPRIML is used only to solve the first linear program $P(\emptyset, \emptyset, \emptyset)$. In order to avoid running phase I of the primal simplex method we provide the LP solver with an *advanced* primal basic feasible solution. Let us denote by $v_1, v_2, \cdots, v_n$ the sequence of the nodes of V in the order that they appear in the *best* tour found by the heuristic. In the first linear program $\mathcal{L} = \emptyset$ and the constraint matrix coincides with A. Any set of columns corresponding to a set of edges E', with $|E'| = n$, such that the partial graph $G = (V, E')$ is spanning and contains one odd cycle forms a basis of A. We take as basis the columns corresponding to the edges $[v_1, v_2], [v_2, v_3], \cdots, [v_{n-2}, v_{n-1}], [v_{n-1}, v_n], [v_n, v_{n-2}]$, that contains the triangle v_{n-2}, v_{n-1}, v_n. The variable $x(v_n, v_1)$ is nonbasic at its upper bound.

Denote by P_1, $P_2, \cdots$ the linear programs that are solved by Algorithm 2.3 in the order in which they are processed and suppose that we have to solve P_k for $k \geq 2$. Let $\overline{x}$ be the optimal solution of P_{k-1} and $\overline{B}$ be the corresponding set of basic variables. Problem P_k differs from P_{k-1} for one of the following reasons:

(a) Some columns have been added to the constraint matrix of P_{k-1}. In this case the variables in $\overline{B}$ form also a basis for P_k and $\overline{x}$ is feasible for the primal and infeasible for the dual problem.

(b) The rows whose corresponding slack variables are in $\overline{B}$ have been removed from the constraint matrix and some rows that are not satisfied by $\overline{x}$ have been added to it. By removing from $\overline{B}$ all slack variables and adding the slack variables of the new constraints, we obtain a set of variables whose corresponding columns form a basis for P_k. The solution $\overline{x}$ is feasible for the dual and infeasible for the primal problem.

(c) $P_{k-1} = P(\mathcal{L}, F_0, F_1)$ and $P_k = P(\mathcal{L}, F'_0, F'_1)$, with $\langle F_0, F_1 \rangle \neq \langle F'_0, F'_1 \rangle$. In this case the constraint matrix does not change (see § 5.5). The variables of $\overline{B}$ form also a basis for P_k. The solution $\overline{x}$ is primal infeasible for P_k and may or may not be feasible for the dual problem.

Even though for case (a) problem P_k could be conveniently solved by XPRIML, we

always use XDUAL because the linear programming problems we deal with exhibit less degeneracy for the dual problem than for the primal one because of the variation in the objective function coefficients of the primal problem.

If the basic feasible solution supplied is not feasible for the dual problem, XDUAL finds a basic feasible solution by switching the nonbasic variables that do not price out correctly from the lower to the upper bound or vice versa. This can be utilized in our case since all of the variables have finite lower and upper bounds. This is clear for the m "structural" variables. For the slack variables we define the upper bound to equal the right-hand side of the corresponding constraint. This is valid because all coefficients of the constraints in $\mathcal{L}$ are nonnegative.

The constraint matrix is stored columnwise in the XMP data structure. For this reason it is difficult to add a row to the constraint matrix. Moreover, the working area used by XMP to store the constraint matrix is shared by the cut generator and hence the matrix representation is destroyed every time the cut generator is executed. Consequently, before going to the LP solver the entire constraint matrix is rebuilt from scratch using the routine XADDAJ and formula (5.3).

5.5. Variable fixing and setting. Let $\overline{x}$ be an *optimal* solution to the LP relaxation of any zero-one program P, cx the objective function to be minimized, and x^* a feasible (integer) solution of P. It is well known that if a component $\overline{x}_i$ has value 1 and its reduced cost r_i satisfies $r_i < -(cx^* - c\overline{x})$, then there is an optimal solution $\hat{x}$ to P having $\hat{x}_i = 1$. Analogously, if a component $\overline{x}_i$ has value zero and its reduced cost satisfies $r_i > cx^* - c\overline{x}$, then there is an optimal solution $\hat{x}$ to P with $\hat{x}_i = 0$. In both cases the variable x_i can be permanently *fixed* to its current value. Fixing variables reduces the number of variables to be handled and tightens the LP relaxation of each linear program of the search tree.

If a variable is fixed to $+1$ it simply never changes its value. If a variable that does not belong to any set of the ordered pairs in the search tree is fixed to zero it is removed from the active set J. Otherwise it is marked as fixed and the LP solver does not consider it any longer as a candidate for entering the basis. Fixing based on reduced cost can be done only if the reduced cost are associated with an *optimal* solution of the current problem $P(\mathcal{L}, F_0, F_1)$. In particular this means that all nonactive variables must have nonnegative reduced cost.

As we are working in a search tree the reduced cost fixing discussed so far applies only to the children of the current parent node $P(\mathcal{L}, F_0, F_1)$ considered. We therefore distinguish between *variable fixing* and *variable setting*. Since all nodes of the search tree are children of the root node the reduced cost of the root node can be used *globally* across the tree to fix variables. To utilize this information we store the objective function value of the root node in RTZLP0 and the reduced cost of the active variables in an array RTREDC that is updated whenever the active set changes, e.g., by setting RTREDC(e) = 0 for a variable x_e that was previously not active. (Setting the number equal to zero does not help to fix such a variable, but it is mathematically correct and computationally "cheap" to do.)

Every time the fixing procedure is executed the value of the gap $cx^* - c\overline{x}$ is stored in FXGAP. The fixing procedure is repeated if any one of the following conditions is satisfied:

(a) The current node is the root of the tree and $cx^* - c\overline{x} <$ FXGAP/2.
(b) A better tour is found and x^* is its incidence vector.
(c) The current node is the root, its current solution is fractional and no inequalities are generated that cut off the fractional solution.

If in case (b) the current node is not the root we use the reduced cost stored in the array RTREDC and the gap $cx^* - \text{RTZLP0}$ to fix variables. To do it otherwise, e.g., with the *current* reduced cost, would be mathematically incorrect.

We run the fixing procedure the first time after problem $P(\emptyset, \emptyset, \emptyset)$ is solved. We scan all variables of E as done in Steps 3 and 4 of Procedure 5.1 by running the edge index k from 1 to m. All inactive variables that cannot be fixed are put in a "reservoir" represented by two arrays RESVFR and RESVTO where the endpoints of each of its variables are stored. As soon as the maximal size of the reservoir is reached the fixing procedure stops and the current values of the index k is assigned to a variable k_0, which is stored.

Every time the fixing procedure is run the variables in the active set are checked first, then the variables in the reservoir are checked and those that can be fixed are removed from it. Finally the edge index k is run from k_0 to m. If an inactive variable corresponding to an edge (i, j), where i and j are obtained from (5.1), is not fixable it is added to an empty location of the reservoir. As soon as the reservoir is full again the fixing procedure stops and the current value of k is assigned to k_0. By using the reservoir the pricing phase of Steps 3 and 4 of Procedure 5.1 can be done more efficiently by analyzing separately the edges of E that are in the reservoir and those whose index is between k_0 and m.

Variable setting is done *locally* in the search tree every time we develop the children of a parent node. We use the reduced cost of the corresponding node $P(\mathcal{L}, F_0, F_1)$ that is first "reconstructed" (see § 6.2).

Both variable fixing and setting are actually done by enforcing equations (2.5). Suppose that the processing of a node $\langle F_0, F_1 \rangle$ is completed and we want to start to solve the node $\langle F_0', F_1' \rangle$. To do so we have to release the enforcing of the variables in $F_0 \cup F_1$. Then we have to set all variables in $F_0' \cup F_1'$ and at the same time we want to keep a basis for the next linear program. This could be done by introducing explicitly equations (2.5) in the constraint set. However, this technique is wasteful as it increases unnecessarily the size of the basis.

If a variable to be set is nonbasic in the current solution we simply mark it as fixed at the desired value. This way the LP solver will keep it forever out of the basis. To do so we use the STATUS array of the LP solver XMP and exploit the way XMP treats artificial variables. If a variable is declared artificial, XMP tries to get it out of the basis as soon as possible and once it becomes nonbasic it will not enter the basis again. An artificial variable has both lower and upper bound equal to zero. Thus if a basic variable x_e in the current solution has to be set or fixed to zero we simply mark it as artificial and as soon as it becomes nonbasic we declare it fixed to its lower bound. If the variable has to be set or fixed to 1 we do as before, but since an artificial variable will eventually get a value of zero, we temporarily subtract the corresponding column of the constraint matrix from the right-hand side vector before calling the LP solver. As soon as the optimal solution is obtained we reset the original right-hand side vector and we add the value c_e to the objective function value.

6. The implementation of the branch-and-cut algorithm. In this section we describe how we have implemented the most important modules of Algorithm 2.3 including the related bookkeeping for the search tree.

6.1. Data structure for the search tree. To represent the search tree we use the following data structure. For every node of the tree we store

(a) The identifier of the parent node.
(b) The index of the branching variable.

(c) The level of the node in the tree (the level is zero for the node $\langle \emptyset, \emptyset \rangle$).
(d) The optimal objective function value of the tightest relaxation obtained for the node.

Two marks are associated with a node. One indicates that the up branch has already been processed and the other indicates that one of the children has been fathomed.

We keep a list of all active nodes ranked in ascending order of their respective objective function values. A node is put in the active list if its best relaxation has a fractional optimal solution and the cut generator has produced no cuts or if it is "paused" (see § 6.3).

6.2. Node selection and processing. The nodes in the tree are processed in the following order. If the ordered list is nonempty the top node is removed from the list and becomes the *current parent node.* If the list is empty the branch-and-cut algorithm stops and the current best solution is the optimal one. The next two nodes to be processed are the up branch of the current parent node first and then the down branch. The strategy of selecting the node with lowest objective function value as the current parent node (best-node search) permits us to keep the search tree as small as possible. Since the algorithm is designed to find an *optimal* solution, best-node search is a reasonable choice for the search strategy.

Once a current parent node $\langle F_0, F_1 \rangle$ has been selected we do not process its children immediately. Rather we process the parent node again until the cut generator stops. This *parent node reconstruction* is done to reproduce the reduced cost vector and the corresponding optimal objective function value of the best relaxation of the parent node. At this point we execute the setting procedure based on the current reduced cost and the current objective function value (see § 5.5). The variables that can be set are added to the sets F_0 and F_1 depending on whether they are set to zero or to 1.

In addition to this conditional setting of variables we set variables on the basis of *logical implications.* For example, if the variables of two edges incident with the same node u are fixed or set to 1, all variables corresponding to the other edges incident with u are set to zero and hence added to F_0.

Every time a node is fathomed we check whether or not the other child of its parent has been already fathomed. In this case the parent node is fathomed as well and this process continues until there are no more nodes having both children fathomed. When this process stops it may happen that the root node of the search tree has one child fathomed. In this case the active child of the root is set to be the new root node of the search tree. If the new root has only one active child, the root is reset again and so on until the root has both of its children active. At this point we reconstruct the best relaxation of the new root and we update RTZLP0 and the array RTREDC. All variables that can be fixed based on reduced cost or on logical implications are permanently fixed and, if $\langle F_0, F_1 \rangle$ is the current root node, the variables in the sets F_0 and F_1 are fixed as well.

6.3. Branch pausing. For large-scale instances of the TSP the upper bound UB found by the heuristic is typically substantially bigger than the length of the optimal tour OPT. As a consequence the optimal objective function value LB of the current linear program $P(\mathcal{L}, F_0, F_1)$ "overshoots" the optimal tour length OPT during the development of a node and the software system keeps on generating cuts until it "hits" the value UB or stops otherwise. In other words, we may just be developing a "wrong branch" of the search tree.

To avoid overshooting we estimate a value TARGET for the optimal tour length OPT. Every time the current value of LB exceeds TARGET the node is *paused* and put back into the active list with its associated current value of LB.

If TARGET is an overestimation of OPT, the paused node is fathomed as soon as a tour with length less than LB is found. Otherwise the paused node eventually becomes the top element of the active list and is selected to be the *current* node.

The estimation of OPT is of course critical for the success of this technique. After substantial experimentation we decided on

$$\text{TARGET} = \lfloor(\text{LBM} - \text{PERC} * \text{LP1})/(1 - \text{PERC})\rfloor + 1, \tag{6.1}$$

where LBM is the minimum of objective function values corresponding to the active nodes (i.e., the one corresponding to the top element of the ordered list), LP1 is the optimal value of $P(\emptyset, \emptyset, \emptyset)$, and PERC is a parameter of value between zero and 1. The setting of TARGET corresponds to the assumption that the value of RATIO (see § 7) is 1 − PERC for the current instance of TSP when OPT is substituted by TARGET. By definition, the value of RATIO is invariant under scaling and translating the distance matrix which is important because of the degree equations in the definition of Q^n. We observed that most of the problems solved in § 7 have the value of RATIO close to 98 for our algorithm and thus we set PERC = 0.02 as a default value. The value of TARGET computed in this way is a *moving target* and changes *adaptively* as LBM increases. In case that TARGET exceeds UB it is, of course, set equal to UB.

6.4. Branching variable selection. Let $\overline{x}$ be the optimal fractional solution of the current node, LB be its corresponding objective function value, and suppose that no more cuts are generated. At this point the node is added to the ordered list of the active nodes and we pick a branching variable x_e having a value $0 < \overline{x}_e < 1$. We consider the choice between the two values $x_e = 0$ or $x_e = 1$—which are the only two possibilities—to be a random event, comparable to flipping a coin. Owing to the fact that in branch-and-cut we do not just impose the simple cuts $x_j \leq 0$ or $x_j \geq 1$—which is what traditional branch-and-bound does—we have chosen to ignore the fine work of the 1960s that went into the development of penalties, i.e., Driebeek penalties, the "pseudo-cost" of IBM's MPSX-MIP/370, etc., concerning the selection of a branching variable, since all of these selection techniques are rather myopic in nature (see, inter alia, the book by Garfinkel and Nemhauser (1972) for a survey of these techniques). Instead, after some experimentation, we decided simply to determine a branching variable such that at least one of the children of the current node is likely to have a final objective function value substantially higher than the current lower bound.

We first determine the largest value $x_e \leq 0.5$ and the smallest value $x_e \geq 0.5$ for $e \in J$. Denote by $x_0 \leq x_1$ the two respective values. We define

$$I = \{e \in J \mid (1 - \text{PAR})x_0 \leq x_e \leq x_1 + \text{PAR}(1 - x_1)\}$$

to be the set of "branchable" variables, where $0 \leq \text{PAR} \leq 1$ is a parameter whose default value equals 0.5. As we wish to solve large-scale problems—meaning that we expect $|I|$ to be a big number—we next analyze the (absolute) values of the objective function coefficients of the variables in I in order to identify a subset $I_2 \subseteq I$ of *outliers*, i.e., of variables with distinguishably large objective function coefficients. It is reasonable to expect that branching on such an outlier variable, no matter what its value in $\overline{x}$, produces a good increase in the objective function.

A set I_2 qualifies to be a set of outliers if $c_i \leq c_j$ for all $i \in I_1 = I - I_2$ and $j \in I_2$, and if

$$\mu_{I_1} + \rho\sigma_{I_1} < \mu_{I_2} - \rho\sigma_{I_2}, \tag{6.2}$$

where μ_{I_k} and σ_{I_k} are the sample mean and standard deviation of c_i for $i \in I_k$ and $k = 1, 2$, respectively. The parameter ρ is set to 3.5 in the algorithm. If (6.2) is satisfied the two samples can be thought of as being "distinguishably apart" by applying Chebyshevs's inequality. If no subset I_2 qualifies as a set of outliers we set $I_2 = I$.

The branching variable is chosen in I_2 as follows: We determine among the values of the variables in I_2 the one that is closest to 0.5 and call this value x_0. The branching variable is the one having the largest objective function coefficient among the variables in I_2 with value $\overline{x}_e$ in the interval $[x_0 - \text{BCTOL}, x_0 + \text{BCTOL}]$, where BCTOL is a parameter the default value of which is set equal to 0.05.

6.5. Inconsistent nodes. Suppose we are solving the problem $P(\mathcal{L}, F_0, F_1)$ and that $J \subseteq E$ is the current set of active variables. If any of the variables in J have been fixed to zero or to 1 we can assume for notational simplicity that these variables are included in F_0 and F_1, respectively.

If the linear problem solved in Step 2 of Procedure 5.1 is not consistent, then the node $\langle F_0, F_1 \rangle$ cannot be fathomed yet because it is possible that it becomes consistent as soon as the set of active variables is increased. Therefore when inconsistency is detected we have to execute a procedure that either enlarges the set J or proves that the problem remains inconsistent over the entire set of variables E unless, possibly, the objective function is permitted to exceed the current upper bound.

Denote by H the constraint matrix and by h the right-hand side vector of the current problem $P(\mathcal{L}, F_0, F_1)$. Let $\overline{u}$ be the current dual feasible solution in the routine XDUAL when the inconsistency is detected, B the corresponding basis of H, and $\overline{z}$ the objective function value. Finally, by x^* we denote the current best tour. The following procedure handles node inconsistency in a mathematically correct way in the context of a *dynamic* branch-and-cut Algorithm 2.3 employing a subprocedure such as Procedure 5.1.

PROCEDURE 6.1. *Input:* c, H, h, J, B^{-1}, $\overline{u}$, $\overline{z}$, x^*.

Step 1. Compute the reduced cost

$$r_e = c_e - \overline{u}H_{\{e\}} \quad \text{for all } e \in E - J.$$

If $r_e \geq 0$ for all $e \in E - J$, go to Step 3.

Step 2. Replace J by $J + \{e \in E - J \mid r_e < 0\}$ and go to Step 10.

Step 3. If $\overline{z} \geq cx^*$, then fathom the node and stop.

Step 4. Set $\overline{h} = B^{-1}h$, clear the marks of the components of $\overline{h}$ and the marks of the variables of E.

Step 5. If all variables in $E - J$ are marked go to Step 9. Otherwise pick a unmarked variable $e \in E - J$ and mark it.

Step 6. If $r_e \geq cx^* - \overline{z}$ go to Step 5, otherwise set $w = B^{-1}H_{\{e\}}$.

Step 7. If for some unmarked component w_i of w, $\overline{h}_i < 0$ and $w_i < 0$ replace J by $J + \{e\}$ and mark all negative components $\overline{h}_j$ such that $w_j < 0$.

Step 8. If all negative components of $\overline{h}$ are marked go to Step 9, otherwise go to Step 5.

Step 9. If any components of $\overline{h}$ are not marked, fathom the node and stop.

Step 10. Solve the problem

$$\max \{cx \mid H_J x \leq h,\ x_{F_0} = \mathbf{0},\ x_{F_1} = \mathbf{1},\ \mathbf{0} \leq x_J \leq \mathbf{1}\}$$

and update $\overline{u}$, B^{-1}, and $\overline{z}$. If the problem is inconsistent go to Step 1, otherwise stop.

In Steps 1 and 2 we check whether or not the current dual vector is dual feasible for the entire problem, and if not we enlarge the set J. Usually this is sufficient to obtain consistency of the primal problem. If this is not the case, in Steps 4–7 we add those variables that may restore the consistency of the primal linear program with an objective function value of at most cx^* and repeat if necessary.

When the procedure stops then the current problem is either consistent and the processing of the node can continue or we can conclude that no feasible integer solution of $P(\mathcal{L}, F_0, F_1)$ has an objective function value less than cx^* and the node can be fathomed.

7. Computational results. The algorithm described in the previous sections has been implemented by us in standard FORTRAN 77. It consists of about 120 routines and the source has approximately 8500 lines of code, not including the comment lines and the LP solver.

Since our software system is designed to handle large-scale problem instances we have tried to save as much memory as possible. To do so we use the same technique as the XMP library: Since FORTRAN does not allow dynamic memory allocation, the memory management for the temporary structures is done by our system. A large array is allocated once and for all that is used to store all temporary structures. The same working area is shared by the heuristic procedure, the LP solver, and the cut generator.

7.1. The test-bed. The system accepts different kinds of input data and in particular, we do not assume a priori any specific metric for the STSP. The problem instance can be given by a list of node coordinates or by the list of all the components of the vector c. The coordinates can be interpreted as Euclidean, pseudo-Euclidean, or as polar coordinates. For the way we compute the distance between points in pseudo-Euclidean coordinates see Padberg and Rinaldi (1987) and for those in polar coordinates see Grötschel and Holland (1988). The distance $c(i,j)$ between two points i and j with Euclidean coordinates x_i, y_i and x_j, y_j, respectively, can be computed in one of the following ways:

(a) $c(i,j) = \left\lfloor \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} + \frac{1}{2} \right\rfloor$.

(b) $c(i,j) = |x_i - x_j| + |y_i - y_j|$.

(c) $c(i,j) = \max\{|x_i - x_j|, |y_i - y_j|\}$.

The distance computed for Euclidean coordinates (case (a)), for pseudo-Euclidean and for polar coordinates is always an integer number. The distance in all other cases is integer if the input data are integer. All the real functions used to compute the distances are taken from the double precision FORTRAN library. Several parameters can be given as input to the algorithm to control its behavior.

All instances of the STSP we used for the testing of our algorithm are taken from either the literature or "real world" problems. The problems with prefix "EIL" are published in Christofides and Eilon (1969) as vehicle routing problems with one depot. We constructed Euclidean TSP instances by taking as nodes of each problem the locations of the customers and the depot. We did the same for problem GIL249, which is published in Gillett and Johnson (1976) as a vehicle routing problem with

249 customers and 14 depots. The ones with prefix "KRO" are Euclidean problems published in Krolak, Felts, and Nelson (1971). Problem LK318P is a Euclidean drilling problem published in Lin and Kernighan (1973), where it is posed as a minimum length Hamiltonian path problem between two given points. The pattern of the points in the plane is composed of three identical modules of 105 nodes, with 3 additional nodes. Problem LK105 is obtained from one of these modules. Problem LK318 is the "Hamiltonian cycle" version of LK318P. The problems with prefix "GH" are geographical problems (except GH442, which is a drilling problem) and are published in Grötschel and Holland (1988). Those with the prefix "ATT" are geographical problems published in Rinaldi and Yarrow (1985) and Padberg and Rinaldi (1987). Problem ALI535 is a geographical problem of 535 airports around the globe that we obtained courtesy of Alitalia Airlines. The problems with prefix "PHI" are Euclidean drilling problems that we obtained by courtesy of Philips, Inc. Finally the problems with prefix "TK" are Euclidean drilling problems that we obtained (in 1979) courtesy of Tektronics, Inc.

With our software system we were able to solve all these problems to optimality. To our knowledge optimal solutions to the problems KRO124, KRO125, KRO126, KRO127, KRO128, LK318P (see Crowder and Padberg (1980)), all problems GH (see Grötschel and Holland (1988)), ATT048 (see Rinaldi and Yarrow (1985)), and LK318P, ATT532, TK1002, TK2392 (see Padberg and Rinaldi (1987)) have been reported in the literature previously.

TK2392 is the largest of the TK problems and has 2392 nodes. To the best of our knowledge this is the largest instance of a symmetric TSP for which an optimal solution is known to date.

For the computational study we used several different computers: A variety of DIGITAL machines of the VAX family, a Control Data machine CYBER 205 and an IBM machine 3090/600. Since the VAX 780 is considerably slower than the other two machines we have executed the experiments with the largest problems only on the CYBER 205 and on the IBM 3090/600. On the other hand, we had much easier access to VAX computers and most of the experiments and the code development were done on VAX machines. Several of the test problems were not available to us when we began this work. The test set for the CYBER 205 is a subset of the test set for the VAX machines (except for three problems) while the test set for the IBM machine contains all of the 42 problems of this study. For the experiments on the VAX machines and the CYBER 205 we used XMP as the LP solver, while for the experiments on the IBM 3090/600 we used a new LP solver written by John Forrest of IBM Research.

In the following we give the results of our experiments in several tables using the following abbreviations:

n:	number of nodes;
LB0:	value of the objective function at the root node;
LP1:	optimal value of $P(\emptyset, \emptyset, \emptyset)$;
UB:	best objective function value found by the heuristic;
OPT:	optimal value of the objective function;
RATIO:	the value of $100(\text{LB0} - \text{LP1})/(\text{OPT} - \text{LP1})$;
CUTS:	total number of cuts generated;
BC:	total number of nodes of the search tree;
MROW:	maximum cardinality of the set of active constraints (including the degree equations);
MCOL:	maximum cardinality of the set of active variables;

NLP: number of executions of the LP solver;
THEU: CPU time in seconds spent in the heuristic;
TRT: total CPU time in seconds spent to process the root node;
TLP: total CPU time in seconds spent in the LP solver (includes the time spent in the pricing procedure and in the set up of the LP solver data structure);
TT: total CPU time in seconds consumed by the whole algorithm.

RATIO equals 100 if the algorithm terminates without branching. If we replace OPT by UB in the definition of RATIO, it is a conservative measure of the progress that the algorithm has made towards proving optimality starting from $P(\emptyset, \emptyset, \emptyset)$; see also Padberg and Hong (1980).

TABLE 4

Problem	n	LB0	RATIO	UB	OPT	CUTS	BC
ATT048	48	10628.0	100.0	10628	10628	37	0
EIL08	51	426.0	100.0	426	426	63	2
KRO070	70	675.0	100.0	675	675	73	0
EIL09	76	538.0	100.0	541	538	21	0
GH096	96	55094.2	95.4	55457	55209	389	4
KRO124	100	21282.0	100.0	21282	21282	168	0
KRO125	100	22136.5	99.7	22141	22141	282	2
KRO126	100	20749.0	100.0	20749	20749	177	0
KRO127	100	21294.0	100.0	21294	21294	139	0
KRO128	100	22018.0	96.5	22100	22068	462	2
EIL10	101	629.0	100.0	630	629	90	0
LK105	105	14379.0	100.0	14379	14379	48	0
TK107	107	44303.0	100.0	44303	44303	127	0
TK124	124	58863.7	98.1	59030	59030	546	2
TK136	136	96546.0	97.0	96998	96772	755	12
GH137	137	69782.5	97.8	69853	69853	353	4
TK144	144	58511.6	99.9	58537	58537	296	2
KRO30	150	26510.8	99.2	26701	26524	578	4
KRO31	150	26114.4	98.9	26182	26130	650	2
TK152	152	73384.0	98.9	75249	73682	1748	34
KRO32	200	29368.0	100.0	29467	29368	814	0
KRO33	200	29437.0	100.0	29699	29437	409	0
GH202*	202	40154.6	99.7	40610	40160	669	4
TK226	226	80369.0	100.0	80690	80369	344	0
GH229*	229	134441.4	97.8	135661	134602	2545	10
GIL249	263	2376.0	98.7	2393	2378	1966	8
TK264	264	49125.7	99.9	49567	49135	122	2
LK318	318	42000.7	99.1	42551	42029	1277	2
LK318P	318	41338.2	99.7	42020	41345	1016	2
GH431*	431	171221.5	97.7	174543	171414	5324	46
GH666a*	666	294050.0	96.8	299225	294357	5061	10
GH666b*	666	294099.0	97.3	294360	294358	4310	20

* run on a VAX 8700

7.2. Computational results on VAX machines. The code has been developed and most of the computational tests have been done on computers of the VAX family. We have used a microVAX II and VAX 8700 of the GBA Academic Computing

Center at New York University, and a VAX 780 of the Centro di Calcolo at Istituto di Analisi dei Sistemi ed Informatica del Consiglio Nazionale delle Ricerche in Rome.

In the VAX implementation the real data are represented with double precision 64-bit numbers and the integer data by 16-bit numbers, except for the cut array of active cuts and the pool array of inactive cuts, which were stored as 8-bit numbers. The computational experiments described in this section were done with the algorithm tuned to solve problems with up to 675 nodes, allowing up to 650 inequalities in the active set and up to 450 inequalities in the pool. The total amount of memory required by the algorithm to run with these parameters is about 3.5 megabytes.

TABLE 5

Problem	n	MROW	MCOL	NLP	THEU	TRT	TLP	TT
ATT048	48	66	127	12	291	21	16	312
EIL08	51	75	116	13	244	25	16	273
KRO070	70	107	171	19	391	58	42	450
EIL09	76	91	183	5	379	16	12	397
GH096	96	158	258	88	1211	677	633	2563
KRO124	100	164	247	30	621	173	127	795
KRO125	100	175	294	46	648	471	316	1136
KRO126	100	160	241	26	647	152	115	801
KRO127	100	146	261	23	721	131	97	853
KRO128	100	184	254	65	659	949	533	1786
EIL10	101	132	247	10	601	70	48	673
LK105	105	131	720	13	829	163	155	993
TK107	107	152	351	25	761	193	167	955
TK124	124	231	443	77	829	814	1173	2492
TK136	136	233	316	121	1055	661	1399	3875
GH137	137	225	314	65	2362	590	450	3077
TK144	144	242	709	59	1047	1075	984	2211
KRO30	150	260	409	75	1297	1465	1114	3135
KRO31	150	255	404	88	1244	1818	1368	3440
TK152	152	307	749	356	1310	1525	13030	17809
KRO32	200	352	559	86	1936	6494	4301	8432
KRO33	200	309	546	41	1989	1014	716	3004
GH202*	202	300	554	79	807	1988	3508	5941
TK226	226	334	1041	53	2259	1683	1465	3944
GH229*	229	413	623	279	916	2179	6497	12208
GIL249	263	442	776	169	2975	2791	7904	14037
TK264	264	319	863	35	3148	826	808	4051
LK318	318	515	1042	97	4976	4342	5988	14213
LK318P	318	507	916	68	4944	5880	4371	12088
GH431*	431	689	1166	560	2510	8081	39646	57029
GH666a*	666	1019	1815	326	5034	4052	48169	65901
GH666b*	666	1004	1819	330	5034	4132	42345	53755

* run on a VAX 8700

In the Tables 4 and 5 we report the computational results on 31 test problems with sizes between 48 and 666 nodes. The heuristic was run 50 times and the default values are used for all the parameters. The parameter XTOLCT was set to 10^{-3}. A total of 12 problems were solved without branching. Problems GH202, GH229, GH431, and GH666 were run on a VAX 8700 at New York University with an "intermediate" version of our code. (With a comparable number of users on both machines

we experienced a speed-up factor of about 5 for the VAX 8700 over the VAX 780.) We ran the algorithm on problem GH666 two times. In the first run (indicated in the tables by GH666a) we computed the distances using single precision arithmetic and subsequent truncation to integers. The optimal tour produced by this run differs from the one reported in Grötschel and Holland (1988) (the symmetric difference of the two tours showed a difference for the European cities) and its length is one unit shorter. In the second run (denoted by GH666b in the tables) we truncated the distances using double precision arithmetic and obtained the same tour as before, but with the same length as in Grötschel and Holland (1988). Thus the problem GH666 does indeed have alternative optima. As we were merely cross-checking the output of the first run with the value published by Grötschel and Holland (1988) we ran our algorithm ceteris paribus, but with an upper bound equal to two units bigger than the figure published there, which reduced the computation times (after the heuristic) by 20% and underscores the importance of a good upper bound for a branch-and-cut optimizer.

7.3. Computational results on the CYBER 205. The CYBER 205 was made available to us by the National Bureau of Standards at Gaithersburg. This machine is a vector computer that can execute parallel operations on arrays. The FORTRAN compiler has the feature of automatically vectorizing some instructions of a program written in standard FORTRAN. We turned this feature off and used exactly the same code as on the VAX with the following changes:

(a) Since the computer can directly access the single bit we stored the vector representing a cut in an array having elements 13 bits long. We used vector instructions to convert a 13-bit number into a regular integer.

(b) We used integers and reals 60 bits long (this is the standard precision of the machine).

(c) We used vector instructions to move in parallel an entire array representing a cut from the pool to the active set and vice versa.

Since we did not use any vector feature (except for point (c)), the machine did not run at its highest speed. As a possible result we observed a speed-up of only about 10 over the VAX 780.

In Tables 6 and 7 we report the computational results on 22 problems with size between 51 and 318 nodes. For these problems the heuristic was executed 10 times. The parameter GIVEUP was set to 10^{-5}. The breakdown of the number of cuts (CUTS) into subtour elimination, comb, and clique-tree inequalities is reported in Padberg and Rinaldi (1990a). Not all of our test problems are included in the CYBER 205 batch because not all of them were available when this experiment was done (December 1986).

In Table 8 we report the results on the three largest of our test problems. The times spent in the heuristic are missing because the heuristic solutions were computed off-line and supplied as input to the algorithm. The heuristic was run 20 times for the problem TK2392 and 50 times for the other two problems. The problems ATT532 and TK2392 were solved with an early version of the algorithm whose cut generator did not yet include the procedure for clique-tree identification. Also, for these problems we did not record some of the statistics and hence in Table 8 some of the data reported in the other tables are missing. The plot of the optimal tours found for the problems TK1002 and TK2392 are shown in the Figs. 8 and 9, respectively, while the plot for the problem ATT532 is published in Padberg and Rinaldi (1987).

As can be seen from Table 8 most of the time consumed by the algorithm is

TABLE 6

Problem	n	LB0	RATIO	UB	OPT	CUTS	BC
EIL08	51	426.0	100.0	426	426	89	0
EIL09	76	538.0	100.0	540	538	29	0
KRO124	100	21282.0	100.0	21282	21282	225	0
KRO125	100	22137.5	99.8	22199	22141	407	2
KRO126	100	20749.0	100.0	20749	20749	166	0
KRO127	100	21294.0	100.0	21362	21294	189	0
KRO128	100	22033.3	97.6	22127	22068	634	2
EIL10	101	629.0	100.0	630	629	72	0
LK105	105	14379.0	100.0	14452	14379	47	0
TK107	107	44303.0	100.0	44438	44303	107	0
TK124	124	58717.9	96.5	59605	59030	595	6
TK144	144	58513.2	99.9	58711	58537	372	2
KRO30	150	26513.5	99.4	26528	26524	630	4
KRO31	150	26117.5	99.1	26483	26130	628	2
TK152	152	73337.6	98.8	75476	73682	1441	18
KRO32	200	29349.3	99.2	29477	29368	1156	2
KRO33	200	29437.0	100.0	29775	29437	356	0
TK226	226	80368.5	99.9	83586	80369	520	2
GIL249	263	2376.2	98.8	2434	2378	2118	10
TK264	264	49124.8	99.9	50381	49135	399	4
LK318	318	41997.9	98.9	43051	42029	1010	6
LK318P	318	41345.0	100.0	42311	41345	1148	0

TABLE 7

Problem	n	MROW	MCOL	NLP	THEU	TRT	TLP	TT
EIL08	51	69	101	15	2	2	1	4
EIL09	76	91	155	9	4	1	1	6
KRO124	100	169	256	39	8	17	12	25
KRO125	100	169	295	60	8	41	28	54
KRO126	100	159	258	29	7	9	6	16
KRO127	100	158	260	36	8	15	10	23
KRO128	100	186	281	79	7	51	40	84
EIL10	101	134	215	16	7	5	3	12
LK105	105	133	289	14	9	5	4	14
TK107	107	146	304	27	8	13	11	21
TK124	124	230	583	118	9	58	124	193
TK144	144	233	687	59	12	93	84	115
KRO30	150	264	367	85	15	148	85	181
KRO31	150	262	420	80	14	113	77	136
TK152	152	296	916	230	14	87	672	875
KRO32	200	357	569	112	25	280	571	848
KRO33	200	295	505	36	24	57	40	81
TK226	226	329	1956	81	28	293	281	344
GIL249	263	454	729	199	35	230	817	1292
TK264	264	360	1136	66	37	91	153	216
LK318	318	490	895	96	57	206	407	664
LK318P	318	504	2944	77	77	934	618	1012

TABLE 8

	ATT532	TK1002	TH2392
Number of nodes	532	1002	2392
Number of variables	141246	501501	2859636
Lower bound at the root	27627.9	258804.7	377854.9
RATIO	94.5	98.7	99.0
Heuristic solution	28150	264874	389325
Optimal solution	27686	259045	378032
LP solver runs	1001	243	394
Active cuts (last node)	220	460	840
Active var. (last node)	1546	2730	4224
Cuts generated	4887	4988	6963
Nodes of the search tree	45	20	53
TRT	650	2826	10448
TLP	—	19104	79425
TT	20759	22692	84037

spent by the LP solver for these large-scale problems. Therefore it can be reasonably expected that a more efficient LP code would decrease the total running time of the algorithm substantially.

7.4. Computational results on the IBM 3090/600. The IBM 3090/600 was made available to us in February of 1989 by the IBM T. J. Watson Research Center in Yorktown Heights under a joint study agreement. The objective of this joint study was to link the linear programming solver OSL written by John Forrest of IBM Research in lieu of the XMP package to the other parts of our software system and to test possible improvements that might result from the use of what is probably the most powerful LP solver existing today. A source code for OSL was not available to us and John Forrest wrote the entire interface between OSL and our software system.

In the IBM implementation the real data are represented with double precision 64-bit numbers and the integer data by 32-bit numbers except for the cut array of active cuts and the pool array of inactive cuts which are stored as 16-bit numbers. The software system was set up to handle up to 5,600 nodes, up to 11,200 inequalities, and up to 560,000 active variables (including slacks). The cut arrays were dimensioned to allow 5,600 active cuts and up to 11,200 inactive cuts in the pool. The IBM 3090/600 is a vector computer, but we used only the modifications described in § 7.3, point (c) to the VAX version of our software system that was suitably changed to accommodate the differences between VAX FORTRAN and IBM FORTRAN. The software system OSL utilizes the vector-processing capabilities of the IBM 3090 series fully.

Due to time limitations—the entire work had to be carried out within three weeks—and in order to keep the experiments of our study comparable, we limited our goal to a rerun of the problems of the previous test set and left experiments with larger problem sizes to future work. The only new problems, those from Philips and Alitalia, fall well within the range of the test problems of our computational study. After some experimentation—and due to John Forrest's enthusiastic help within the three week time limit—we succeeded in having an operational software system running on the IBM 3090/600 for the large-scale TSPs considered here and the results of our runs are displayed in Tables 9 and 10. The heuristic was executed 10 times on each of the problems except for the two last runs, which were excuted twice. All other parameters were set at their default values described in this paper except XTOLCT

TABLE 9

Problem	n	LB0	RATIO	UB	OPT	CUTS	BC
ATT048	48	10628.0	100.0	10628	10628	43	0
EIL08	51	426.0	100.0	427	426	98	2
KRO070	70	675.0	100.0	679	675	108	0
EIL09	76	538.0	100.0	542	538	18	0
TK076	76	107466.5	92.4	108159	108159	3853	66
GH096	96	55019.8	92.4	55733	55209	553	6
KRO124	100	21282.0	100.0	21282	21282	208	0
KRO125	100	22141.0	100.0	22199	22141	300	0
KRO126	100	20749.0	100.0	20749	20749	176	0
KRO127	100	21294.0	100.0	21442	21294	207	0
KRO128	100	22038.3	97.9	22106	22068	613	2
EIL10	101	629.0	100.0	630	629	59	0
LK105	105	14379.0	100.0	14379	14379	46	0
TK107	107	44303.0	100.0	44303	44303	120	0
TK124	124	58867.3	98.2	59246	59030	335	2
TK136	136	96546.0	97.0	96998	96772	788	10
GH137	137	69795.3	98.2	70517	69853	385	2
TK144	144	58493.6	99.8	58809	58537	350	2
KRO30	150	26513.2	99.4	26744	26524	609	4
KRO31	150	26117.8	99.1	26350	26130	732	2
TK152	152	73374.0	98.9	75249	73682	1388	18
KRO32	200	29359.9	99.6	29536	29368	1191	2
KRO33	200	29437.0	100.0	29746	29437	387	0
GH202	202	40154.1	99.7	40644	40160	497	2
PHI224	224	141779.0	100.0	142141	141779	283	0
TK226	226	80369.0	100.0	80690	80369	364	0
GH229	229	134395.6	97.1	136323	134602	2532	10
GIL249	263	2376.2	98.9	2400	2378	1604	2
TK264	264	49128.4	99.9	51781	49135	147	2
PHI298	298	232271.0	99.9	239293	232356	971	4
TK299	299	481490.1	98.6	488023	481948	3507	8
LK318	318	42000.9	99.1	42551	42029	1402	4
LK318P	318	41341.3	99.9	42020	41345	1114	2
GH431	431	171162.6	97.0	173111	171414	4047	24
PHI435	435	297197.0	100.0	308694	297197	2261	0
TK439	439	106923.2	97.8	109234	107217	17734	120
GH442	442	5065.7	94.9	5133	5069	7339	92
ATT532	532	27655.6	97.1	27985	27686	16987	106
ALI535	535	202207.4	99.1	207537	202310	1895	6
GH666	666	294080.3	97.1	302040	294358	5073	20
TK1002	1002	258856.6	99.0	266437	259045	11020	18
TK1002*	1002	258860.1	99.0	265513	259045	9164	12
TK2392	2392	377986.5	99.8	390474	378032	10238	2
TK2392*	2392	378026.6	99.9	389374	378032	9850	2

* The heuristic was executed 50 times.

(see § 4), which was set equal to 10^{-7}. The plot of the optimal tour found for the problem ALI535 is shown in Fig. 10.

8. Final remarks. The results of our computational study confirm our hypothesis that the branch-and-cut approach to combinatorial optimization based on the

TABLE 10

Problem	n	MROW	MCOL	NLP	THEU	TRT	TLP	TT
ATT048	48	66	114	16	3	2	2	5
EIL08	51	86	97	19	2	3	2	5
KRO070	70	121	152	18	3	3	2	6
EIL09	76	88	151	5	4	1	1	4
TK076	76	679	246	771	4	28	372	883
GH096	96	187	222	126	9	19	24	63
KRO124	100	170	235	31	5	7	5	12
KRO125	100	180	249	46	6	14	9	19
KRO126	100	163	227	28	6	6	4	11
KRO127	100	159	227	36	6	8	6	14
KRO128	100	250	254	76	6	34	17	44
EIL10	101	135	201	11	5	2	2	7
LK105	105	134	310	12	7	2	2	9
TK107	107	151	477	22	7	5	5	13
TK124	124	206	721	60	7	19	15	31
TK136	136	259	276	110	9	31	27	71
GH137	137	242	289	67	18	23	16	44
TK144	144	258	879	55	11	21	18	34
KRO30	150	304	343	75	12	32	23	56
KRO31	150	277	369	78	12	43	24	58
TK152	152	323	1347	207	12	24	103	195
KRO32	200	389	478	116	20	107	78	229
KRO33	200	318	501	32	17	16	10	33
GH202	202	338	450	60	33	30	21	71
PHI224	224	309	2933	34	22	21	17	43
TK226	226	335	993	40	21	23	19	44
GH229	229	531	535	219	37	111	211	650
GIL249	263	451	597	117	27	41	64	187
TK264	264	324	779	32	28	19	18	49
PHI298	298	510	1229	95	35	62	70	130
TK299	299	801	715	249	35	310	569	1337
LK318	318	616	939	113	46	87	92	292
LK318P	318	512	782	75	44	139	53	192
GH431	431	831	1022	318	110	125	607	1493
PHI435	435	790	1225	93	84	420	193	504
TK439	439	1547	1751	1370	66	320	7869	13412
GH442	442	700	860	864	54	43	629	1702
ATT532	532	2273	1278	1098	109	547	8731	19805
ALI535	535	822	1302	136	157	173	266	541
GH666	666	1185	1417	293	202	280	1193	2311
TK1002	1002	2059	2365	334	323	1120	5225	11714
TK1002*	1002	1740	2512	250	1618	1265	4571	11379
TK2392	2392	3908	5273	112	1543	5465	5890	9515
TK2392*	2392	3833	5635	96	8004	7390	4929	15753

* The heuristic was executed 50 times.

linear description of polyhedra can be used to solve problems of substantially bigger size to optimality than was considered to be possible only a few years ago. Nor is there any reason to believe that the results reported here constitute a "barrier" to the size of problems that are amenable to exact optimization. On the contrary, we feel confident to state that polyhedral theory is good practice and that research advances

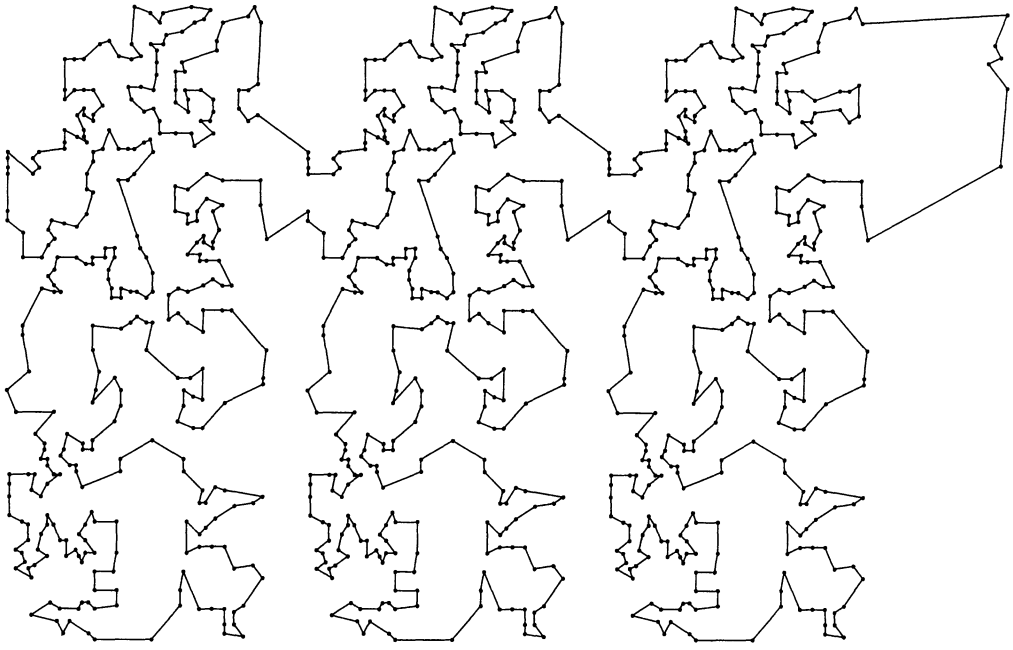


FIG. 8. *Optimal tour for* TK1002 *of length* 259,045.

along polyhedral lines will push computational possibilities substantially beyond the results of this study—provided, of course, that this line of research does not lose its current computational flavor.

There are several reasons for our optimism that are evident from the tables of § 7—like the consistently small number of nodes of the search trees—and the most striking one is due to the dramatic improvement in running times that we experienced when using OSL as the LP solver: While the CYBER 205 runs of the two largest problems TK1002 and TK2392 of our study required about 7 hrs., 18 mins. and 27 hrs., 20 mins., respectively, the IBM 3090/600 runs required about 3 hrs., 10 mins. and 2 hrs., 40 mins., respectively. More important than the reduction in absolute running time, however (which requires a comparison of different computing machinery), is the reduction of the relative proportion of the LP times (TLP) to the overall running times (TT) of the two problems. While on the CYBER 205 linear-programming related calculations (set-up, optimization, factorization, pricing, etc.) accounted for over 90% of the total running times, the correponding proportions are now less than 40% for TK1002 and less than 60% for TK2392. For problem ATT532 which we solved originally in about 60 hours on a VAX 780 (see Padberg and Rinaldi (1987)), the reduction in running time for the IBM 3090/600 run over the CYBER 205 run is negligible. However, the proportion of LP time to total running time is less than 50% as well. Thus we can conclude that, as LP solvers grow faster and faster, advances in algorithmic constraint generation are guaranteed to improve the total running times of branch-and-cut algorithms for combinatorial optimization problems. As stated in Padberg and Rinaldi (1990a), in the early phase of this study we experienced a breakthrough in the reduction of running times with the implementation of the minimum

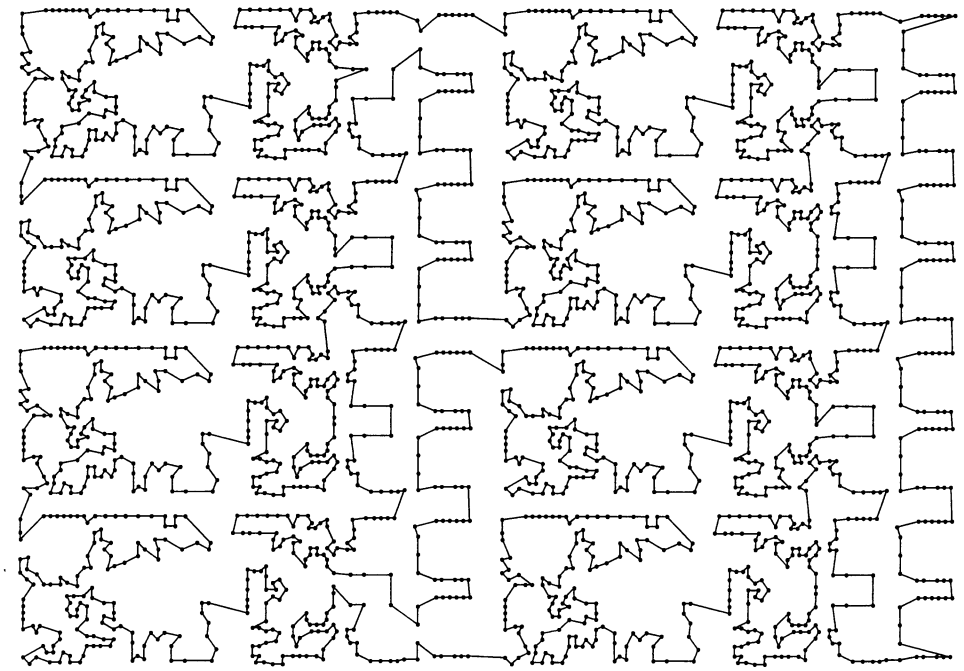


FIG. 9. *Optimal tour for* TK2392 *of length* 378,032.

cut algorithm described there. Similar breakthroughs are needed to improve the speed with which we can find violated 2-matching, comb, and clique-tree inequalities, as well as the new inequalities that have been found only recently and for which the constraint identification problem has so far not even been addressed. To give a concrete example, in problem TK1002 the constraint generation consumed about 1 hr., 30 mins. of CPU time of which less than 4 mins. were used to identify a total of 3,881 subtour elimination constraints, about 15 mins. were spent on scanning the inactive cuts in the pool and the rest of 1 hr., 10 mins. on finding a total of 6,613 comb and 526 clique-tree constraints. Evidently, the comb identification must be sped up, and in our estimation this is possible. For the resolution of truly large-scale STSPs this calls for much research and hard work for years to come. In addition to the algorithmic work that needs to be done, there are, however, a number of other questions of a more strategic flavor that await answers. One of these, addressed in § 4.3, concerns the problem of *finding the right cut.* Another one of these questions is far simpler to put: How large should the pool array be dimensioned in relation to the number of nodes of the problem? In the IBM version this parameter was set equal to 11,200 as storage on the 3090/600 presented no problem to us for the sizes considered here. As a result thousands of inactive cuts were stored and had to be checked individually for violation at every iteration of the constraint generation phase. For the run of problem ATT532 this exhaustive search accounts for 30% of the total time for the cut generation phase which corresponds to about 50 mins. of CPU time. (For problem TK439 this percentage was about 60%, while for the small, but difficult problem TK076 it was over 25%.) Evidently, the size of the pool must be set much more judiciously than we did in the IBM version; but then the time limitation of three weeks prohibited us from



FIG. 10. *Optimal tour for* ALI535 *of length* 202,310.

investigating this issue further and we left it for future work.

As the computational study comprises a substantial number of test problems we ran a least-squares estimate on the various running times and relative error measures for the runs on the IBM 3090/600 to summarize the empirical performance of the branch-and-cut algorithm for the symmetric traveling salesman problem. In Table 11 we give the least-squares estimates of the running times of the heuristic THEU(n), the LP times TLP(n), the cut generation times TCUT(n), and the overall running times TT(n) as a function of the number of the nodes n of the STSP. The cut generation times are not included in Table 10; they are essentially equal to the total times (TT) minus the sum of the times for the heuristic (THEU) and the LP times (TLP). Even though a relative error measure must be viewed with caution as—due to the degree equations—it is not invariant under a *translation* of the data, we have computed two such measures. Running a linear regression on the relative error (UB − OPT)/OPT of the *best* (UB) of the 10 heuristic tour lengths over the optimal tour lengths (OPT) we find the formula $0.67 + 0.0016n$ when expressed in percentage and across the entire sample of 42 test problems the average relative error equals 1.14% with a standard deviation of 1.19%. Doing likewise for the relative error (OPT − LB0)/LB0 of the optimum tour length (OPT) over the lower bound at the root node (LB0) we find the formula $0.0095 - 0.00003n$ when expressed in percentage and across the entire sample the average error equals 0.086% with a standard deviation of 0.13%. The overall empirical performance thus looks very satisfactory indeed for a problem that is as complex as the symmetric traveling salesman problem and as noted before, the times for cut generation stand out as the most time-consuming factor if a powerful LP solver such as OSL is used. But then comparatively little time has been spent to date on finding *fast* algorithms for automatic constraint generation so that much progress seems possible in this field. Such progress is bound to produce a faster algorithm that is capable of optimizing much larger problem sizes than the ones considered in this computational study in "reasonable" computation times or—to paraphrase the conclusion of Padberg and Grötschel (1985)—the problem with 2392 cities should not be the end of the ongoing saga of the symmetric traveling salesman.

TABLE 11

Running time	Estimate
THEU(n)	$2.4 \times 10^{-3}\, n^{1.71}$
TLP(n)	$8.1 \times 10^{-5}\, n^{2.51}$
TCUT(n)	$1.2 \times 10^{-5}\, n^{2.77}$
TT(n)	$3.5 \times 10^{-4}\, n^{2.41}$

Acknowledgments. During the course of this study we received generous support from many institutions and people associated with the computer centers where this work was carried out. We thank all of them. Karla Hoffman of George Mason University provided us with access to the CYBER 205 of the National Bureau of Standards and thus we share the blame for his being one of the heaviest users of this facility during several months in 1986–87. During the more recent phase of this study, Karla helped us to pinpoint some strategic issues in the implementation of the branch-and-cut algorithm such as target setting and branching variable selection. John Forrest of IBM Research wrote the interface with his LP solver OSL that allowed us to do the last part of the computational study and was most helpful in getting us acquainted

with the latest generation of IBM computers and software. We are indebted to both of them.

We thank Roy Marsten of the University of Arizona for making the source code of XMP available to us.

We gratefully acknowledge Alitalia Airlines, Philips, Inc., and Tektronics, Inc. for making data sets available to us.

We are most grateful for the generous support and assistance given to us by our colleagues of the National Bureau of Standards.

We gratefully acknowledge the support we received from GBA Academic Computing Center at New York University and the Istituto di Analisi dei Sistemi ed Informatica del Consiglio Nazionale delle Ricerche in Rome.

We are grateful to IBM Corporation for making their computing machinery available to us.

REFERENCES

J. R. ARAQUE (1989), *Contributions to the polyhedral approach to vehicle routing*, Ph.D. thesis, SUNY at Stony Brook, Stony Brook, NY.

S. C. BOYD AND W. H. CUNNINGHAM (1988), *Small traveling salesman polytopes*, Preprint, Carleton University, Ottawa.

P. M. CAMERINI, L. FRATTA, AND F. MAFFIOLI (1975), *On improving relaxation methods by modified gradient techniques*, Math. Programming Stud., 3, pp. 26–34.

N. CHRISTOFIDES AND S. EILON (1969), *An algorithm for the vehicle-dispatching problem*, Oper. Res. Quart., 20, pp. 309–318.

H. CROWDER AND M. PADBERG (1980), *Solving large-scale symmetric traveling salesman problems to optimality*, Management Sci., 26, pp. 495–509.

G. B. DANTZIG, D. R. FULKERSON, AND S. M. JOHNSON (1954), *Solution of a large-scale travelling salesman problem*, Oper. Res., 2, pp. 393–410.

———(1959), *On a linear-programming combinatorial approach to the traveling-salesman problem*, Oper. Res., 7, pp. 58–66.

R. S. GARFINKEL AND G. L. NEMHAUSER (1972), *Integer Programming*, John Wiley, New York.

B. E. GILLETT AND J. G. JOHNSON (1976), *Multi-terminal vehicle-dispatch algorithm*, Omega, 4, pp. 711–718.

M. GRÖTSCHEL (1977), *Polyedrische Charakterisierungen kombinatorischer Optimierungsprobleme*, Verlag Anton Hain, Meisenheim-am-Glan, FRG.

———(1980), *On the symmetric travelling salesman problem: Solution of a* 120*-city problem*, Math. Programming Study, 12, pp. 61–77.

M. GRÖTSCHEL AND O. HOLLAND (1988), *Solution of large-scale symmetric travelling salesman problems*, Report 73, Institut für Mathematik, Universität Augsburg, Augsburg, FRG.

M. GRÖTSCHEL, L. LOVÁSZ, AND A. SCHRIJVER (1981), *The ellipsoid method and its consequences in combinatorial optimization*, Combinatorica, 1, pp. 169–197.

M. GRÖTSCHEL AND M. PADBERG (1978), *On the symmetric traveling salesman problem: Theory and computation*, in Optimization and Operations Research, R. Henn et al., eds., Springer-Verlag, Berlin, pp. 105–115.

———(1979a), *On the symmetric travelling salesman problem* I*: Inequalities*, Math. Programming, 16, pp. 265–280.

———(1979b), *On the symmetric travelling salesman problem* II*: Lifting theorems and facets*, Math. Programming, 16, pp. 281–302.

———(1985), *Polyhedral theory*, in The Traveling Salesman Problem, E. L. Lawler et al., eds., John Wiley, Chichester, UK, pp. 251–305.

M. GRÖTSCHEL AND W. R. PULLEYBLANK (1986), *Clique tree inequalities and the symmetric travelling salesman problem*, Math. Oper. Res., 11, pp. 537–569.

K. HELBIG HANSEN AND J. KRARUP (1974), *Improvements of the Held-Karp algorithm for the symmetric traveling-salesman problem*, Math. Programming, 7, pp. 87–96.

M. HELD AND R. M. KARP (1971), *The traveling-salesman problem and minimum spanning trees: part* II, Math. Programming, 1, pp. 6–25.

A. J. HOFFMAN AND P. WOLFE (1985), *History*, in The Traveling Salesman Problem, E. L. Lawler et al., eds., John Wiley, Chichester, UK, pp. 1–15.

K. HOFFMAN AND M. PADBERG (1985/6), *LP-based combinatorial problem solving*, Ann. Oper. Res., 4, pp. 145–194.

P. D. KROLAK, W. FELTS, AND J. H. NELSON (1971), *A man-machine approach towards solving the traveling salesman problem*, Comm. ACM, 14, pp. 327–334.

E. L. LAWLER, J. K. LENSTRA, A. H. G. RINNOOY KAN, AND D. B. SHMOYS, EDS. (1985), *The Traveling Salesman Problem*, John Wiley, Chichester, UK.

T. G. LEWIS AND W. H. PAYNE (1973), *Generalized feedback shift register pseudorandom number algorithm*, J. Assoc. Comput. Mach., 20, pp. 456–468.

S. LIN AND B. W. KERNIGHAN (1973), *An effective heuristic algorithm for the traveling salesman problem*, Oper. Res., 21, pp. 498–516.

R. MARSTEN (1981), *The design of the* XMP *linear programming library*, ACM Trans. Math. Software, 7, pp. 481–497.

P. MILIOTIS (1978), *Using cutting planes to solve the symmetric traveling salesman problem*, Math. Programming, 15, pp. 177–188.

D. NADDEF AND G. RINALDI (1988a), *The symmetric traveling salesman polytope and its graphical relaxation: Composition of valid inequalities*, Report RR 719-M, ARTEMIS-IMAG, Grenoble, to appear in Math. Programming.

———(1988b), *The graphical relaxation: A new framework for the symmetric traveling salesman polytope*, Report R.244, IASI-CNR, Rome, to appear in Math. Programming.

———(1988c), *The symmetric traveling salesman polytope: New facets from the graphical relaxation*, Report R.248, IASI-CNR, Rome.

———(1988d), *The crown inequalities for the symmetric traveling salesman polytope*, Report R.249, IASI-CNR, Rome, to appear in Math. Oper. Res.

G. L. NEMHAUSER AND L. A. WOLSEY (1988), *Integer and combinatorial optimization*, John Wiley, Chichester, UK.

M. PADBERG (1973), *A note on the convergence of integer programming algorithms using intersection cuts*, Mimeo 1970, Preprint I/73–46, Internat. Inst. of Management, Berlin.

———(1971), *Essays in integer programming*, Ph.D. thesis, GSIA, Carnegie-Mellon University, Pittsburgh, PA.

———(1973), *On the facial structure of set packing polyhedra*, Math. Programming, 5, pp. 199–215.

M. PADBERG AND M. GRÖTSCHEL (1985), *Polyhedral computations*, in The Traveling Salesman Problem, E. L. Lawler et al., eds., John Wiley, Chichester, UK, pp. 307–360.

M. PADBERG AND S. HONG (1980), *On the symmetric traveling salesman problem: A computational study*, Math. Programming Study, 12, pp. 78–107.

M. PADBERG AND M. R. RAO (1981), *The Russian method for linear inequalities* III*: bounded integer programming,* GBA Report 81–39, New York University, New York.

———(1982), *Odd minimum cut-sets and b-matchings*, Math. Oper. Res., 7 pp. 67–80.

M. PADBERG AND G. RINALDI (1987), *Optimization of a* 532*-city symmetric traveling salesman problem by branch-and-cut*, Oper. Res. Letters, 6, pp. 1–7.

———(1989), *A branch-and-cut approach to a traveling salesman problem with side constraints*, Management Sci., 35, pp. 1393–1412.

———(1990a), *Facet identification for the symmetric traveling salesman polytope*, Math. Programming, 47 pp. 219–257.

———(1990b), *An efficient algorithm for the minimum capacity cut problem*, Math. Programming, 47, pp. 19–36.

G. RINALDI AND L. A. YARROW (1985), *Optimizing a* 48*-city traveling salesman problem: A case study in combinatorial problem solving*, GBA Report 85–42, New York University, New York.

A. E. STORY AND H. M. WAGNER (1963), *Computational experience with integer programming for job-shop scheduling*, in Industrial Scheduling, J. F. Muth and G. F. Thomson, eds., Prentice-Hall, Englewood Cliffs, NJ, pp. 207–219.

C. A. TRAUTH, JR., AND R. E. WOOLSEY (1969), *Integer programming: A study in computational efficiency*, Management Sci., 15, pp. 481–493.

H. M. WAGNER, R. J. GIGLIO, AND R. G. GLASER (1964), *Preventive maintenance scheduling by mathematical programming*, Management Sci., 10, pp. 316–334.

H. WEYL (1935), *Elementare Theorie der konvexen Polyeder*, Commentarii Mathematici Helvetici, 7, pp. 509–533.